

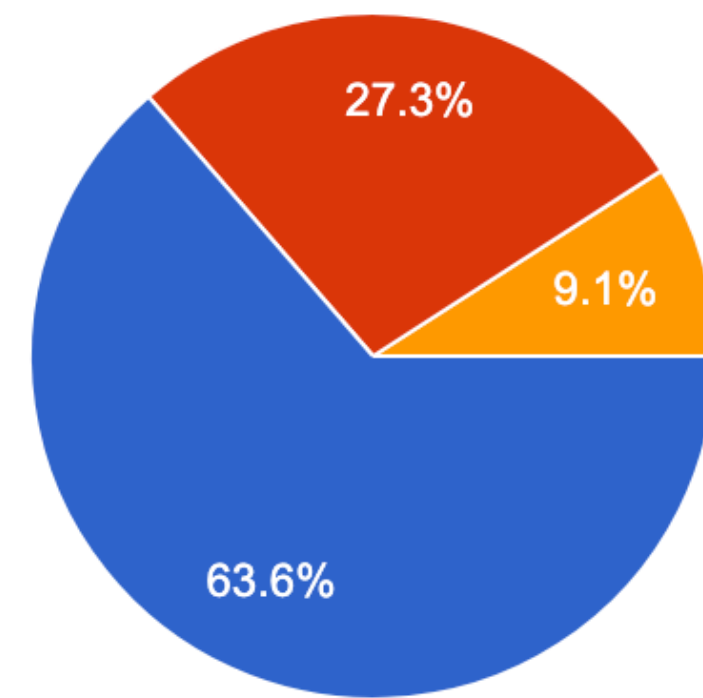
Machine Learning for **Very** Newbies — A Practical Guide

Yongik Jang



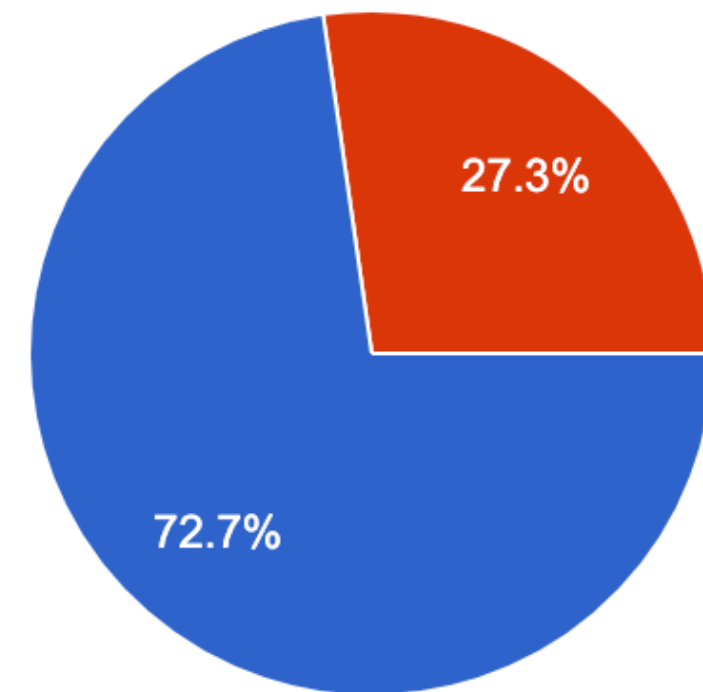
JCP HEP School
Chungnam National University, Daejeon, Korea
19 September 2026

What is your experience with Machine Learning?



- No prior experience
- I have completed tutorials or online courses
- I have applied ML to research projects

What level of hands-on session would you prefer?



- Beginner (basic concepts and step-by-step implementation)
- Intermediate (practical applications and model training)
- Advanced (state-of-the-art methods and research-oriented examples)

ML for Very Newbies

A Practical Guide

Simple ideas. Real examples. You can do it! 😊

Computer Vision
Physics
Generative AI
and beyond...

Welcome
to
Machine
Learning!

No prior experience
required! 😊

I don't know
anything about ML...
Is that okay?

Yep!
That's exactly
why you're here.
We'll learn
together!

Let's
go!

Cool things
ahead!

More
exciting
things...

1
Perceptron



2
MLP



3
Nonlinearity



4
Loss &
Optimization



5
MNIST

3

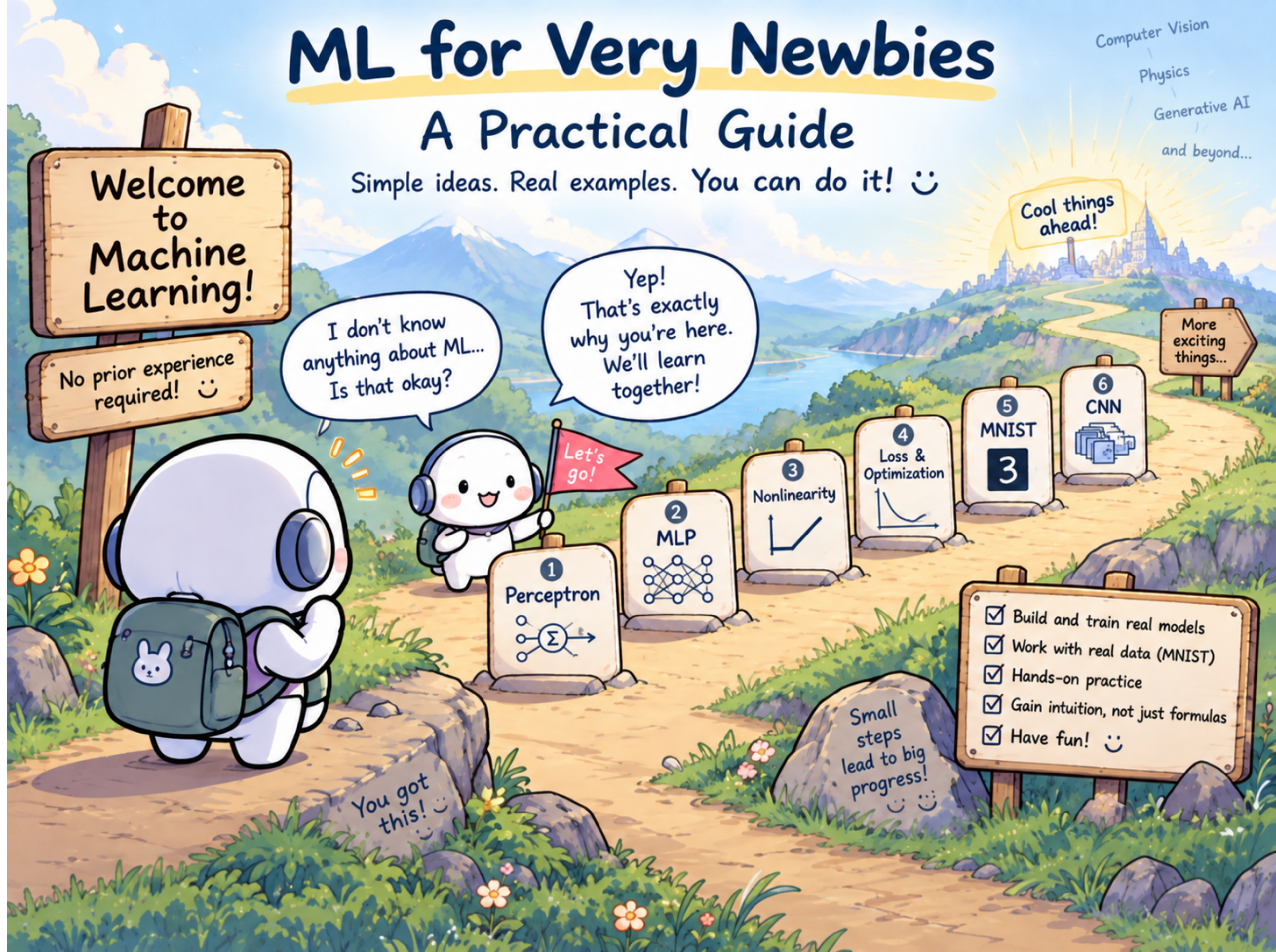
6
CNN



- ✓ Build and train real models
- ✓ Work with real data (MNIST)
- ✓ Hands-on practice
- ✓ Gain intuition, not just formulas
- ✓ Have fun! 😊

Small
steps
lead to big
progress!
😊😊

You got
this! 😊

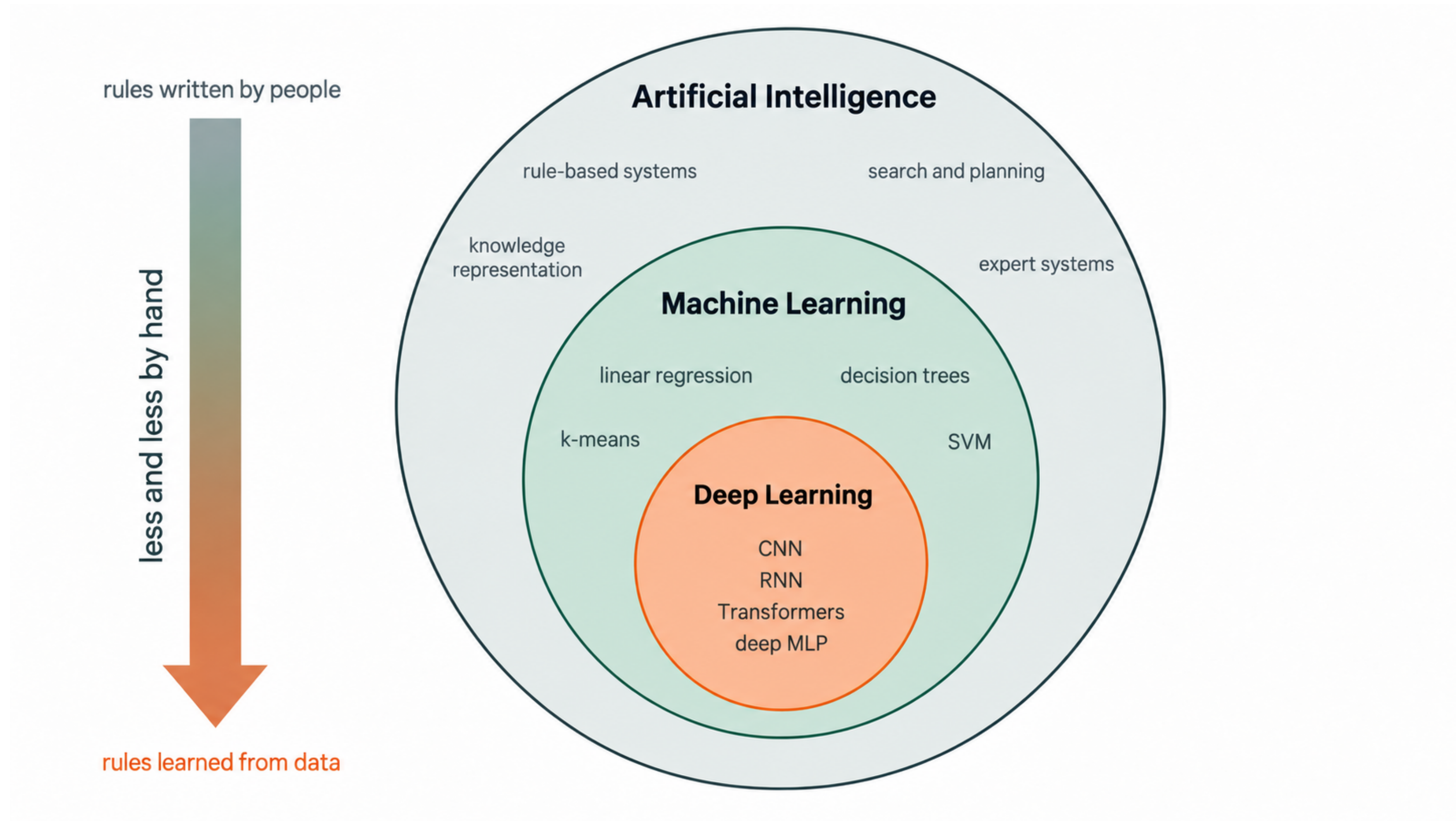


Outline

- Introduction
- Building Blocks
- Training Neural Network
- Hands-On MNIST
- Convolutional Neural Network
- Epilogue

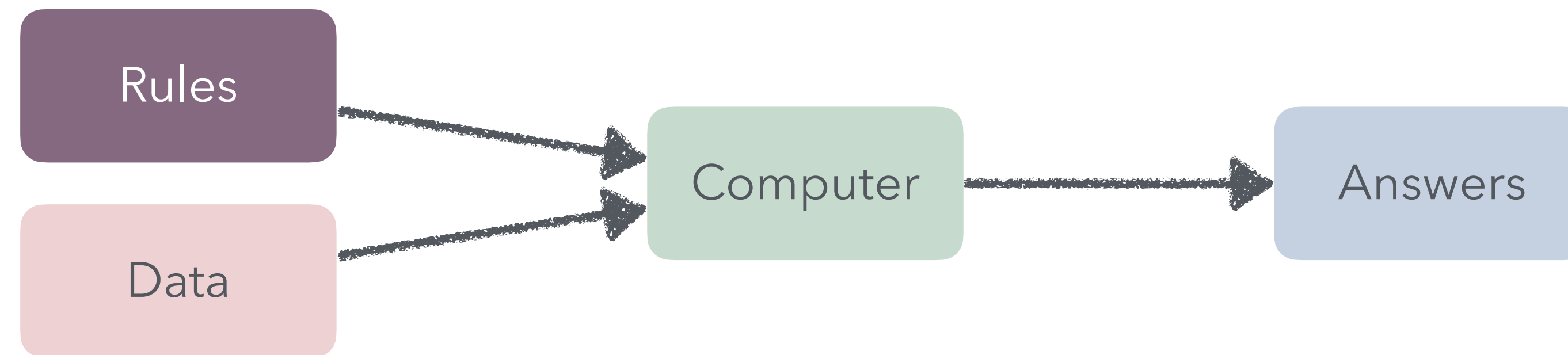
Introduction

What Is Machine Learning?

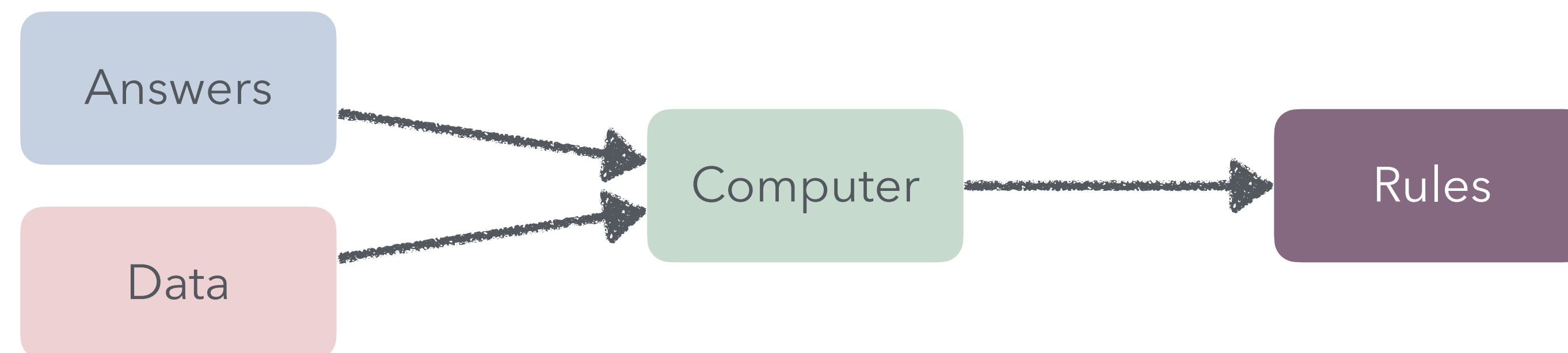


What Is Machine Learning?

- Classical programming



- Machine learning



How to Learn?

Supervised

given: x and y

Learn a mapping from input to a known target.

Typical tasks

- Classification – which digit?
- Regression – what energy?

In HEP

- Jet tagging, BSM searches

Unsupervised

given: x only

Learn the structure of the data itself.

Typical tasks

- Density estimation – what does typical data look like?
- Generative modeling – make new samples that look like the data

In HEP

- Anomaly detection

Reinforcement

given: a reward signal

Learn a policy by acting and seeing what the environment returns.

Typical tasks

- Sequential decisions
- Control

In HEP

- Accelerator beam tuning

(<https://doi.org/10.1038/s41598-024-66263-y>)

There are other ways to learn beyond these classes: self-supervised, semi-supervised...

How to Learn?

Today: supervised only!



Supervised

given: x and y

Learn a mapping from input to a known target.

Typical tasks

- Classification – which digit?
- Regression – what energy?

In HEP

- Jet tagging, BSM searches

Unsupervised

given: x only

Learn the structure of the data itself.

Typical tasks

- Density estimation – what does typical data look like?
- Generative modeling – make new samples that look like the data

In HEP

- Anomaly detection

Reinforcement

given: a reward signal

Learn a policy by acting and seeing what the environment returns.

Typical tasks

- Sequential decisions
- Control

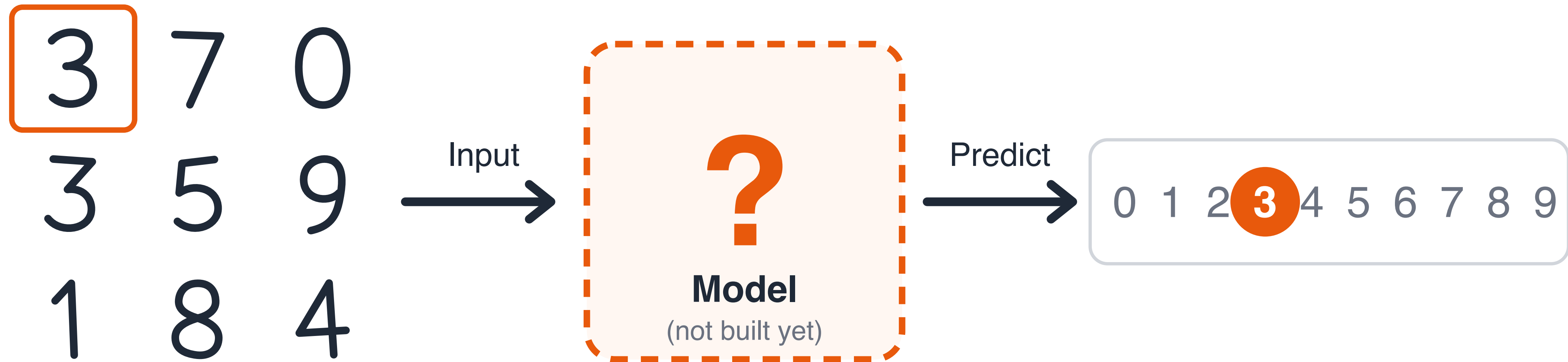
In HEP

- Accelerator beam tuning

(<https://doi.org/10.1038/s41598-024-66263-y>)

There are other ways to learn beyond these classes: self-supervised, semi-supervised...

Today's Problem: MNIST



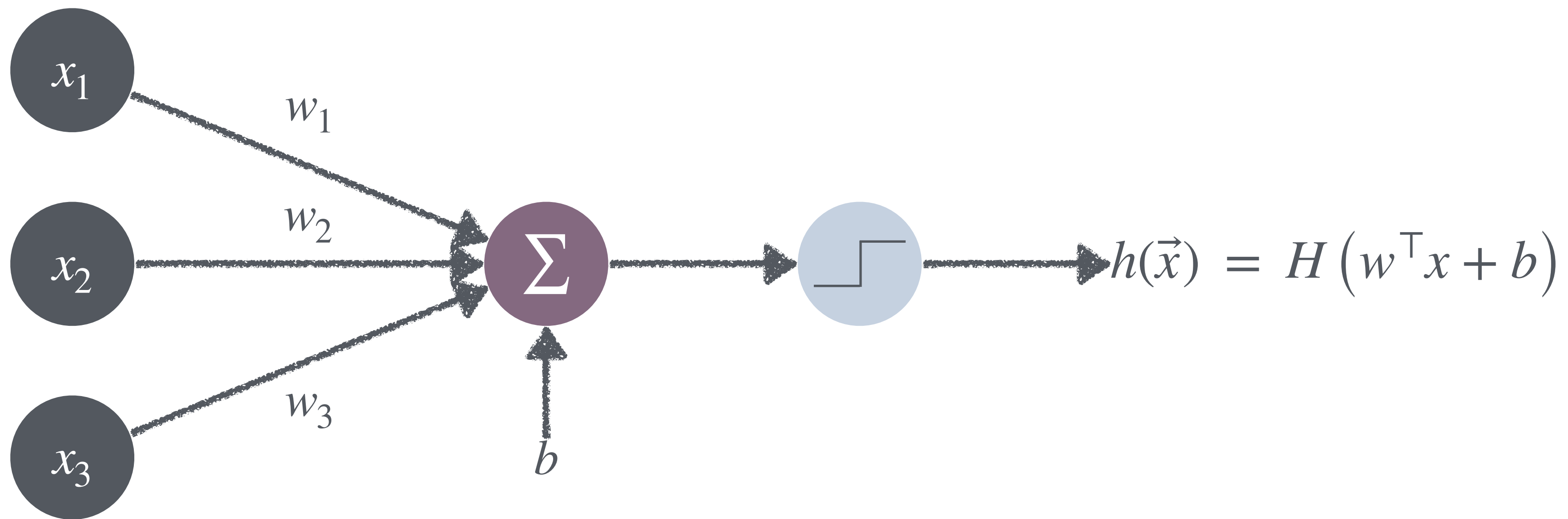
28 × 28 grayscale images

Our task: one image → one of 0–9

Building Blocks

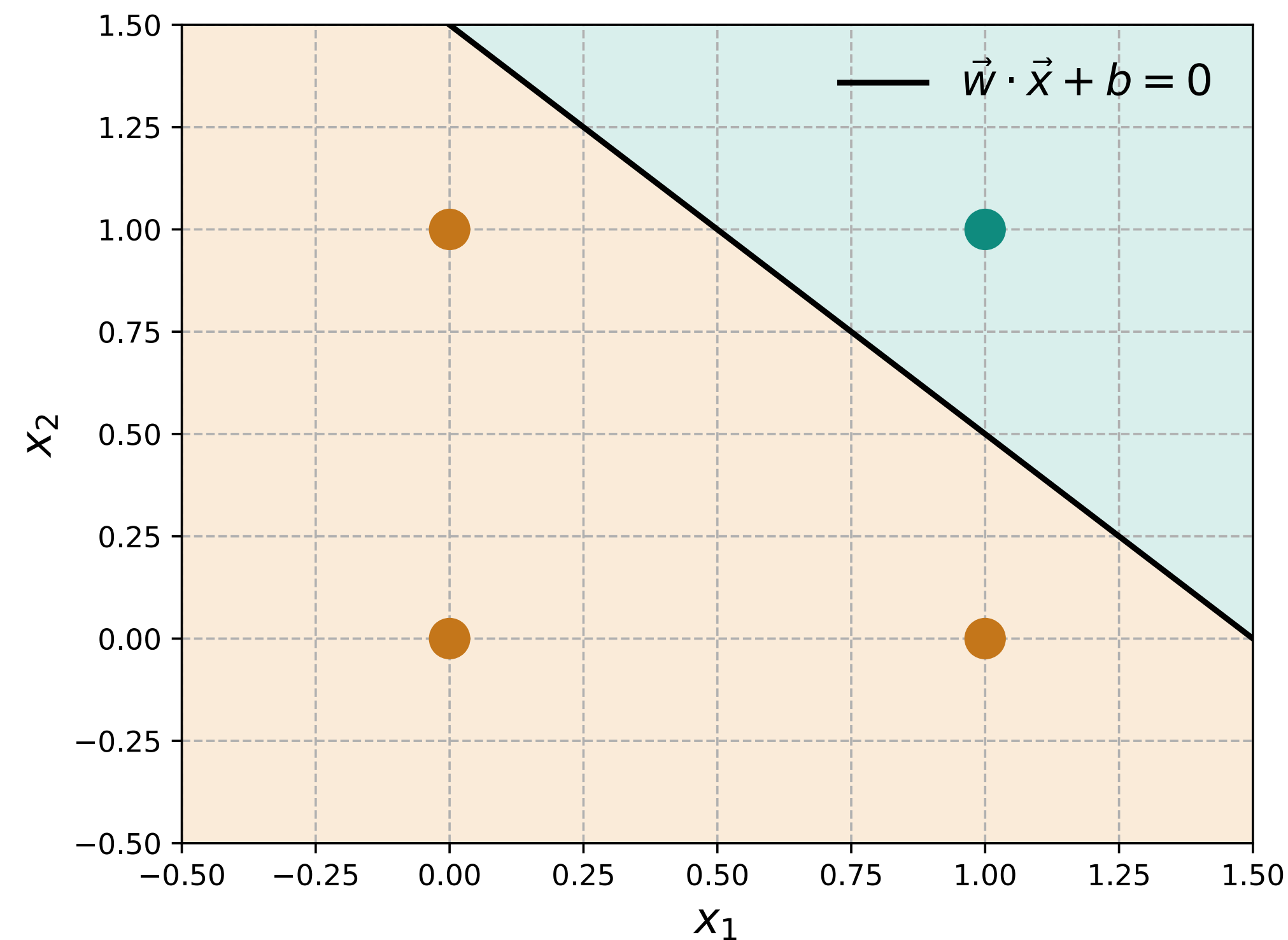
What Is a Neural Network Made Of?

The Unit: Perceptron



What Is a Neural Network Made Of?

The Unit: Perceptron

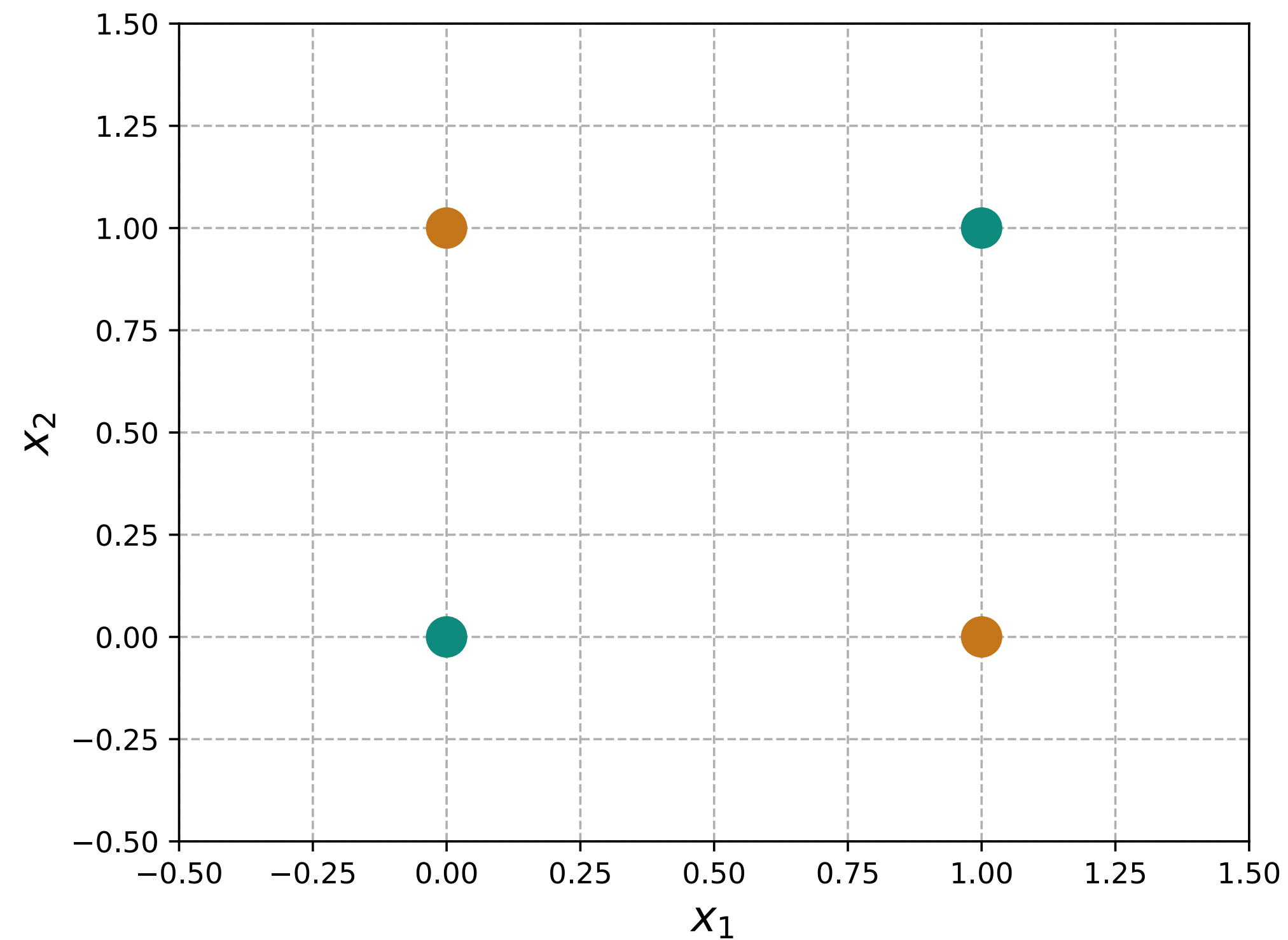


A single perceptron draws one straight line (in higher dimensions: one hyperplane).

What Is a Neural Network Made Of?

The Unit: Perceptron

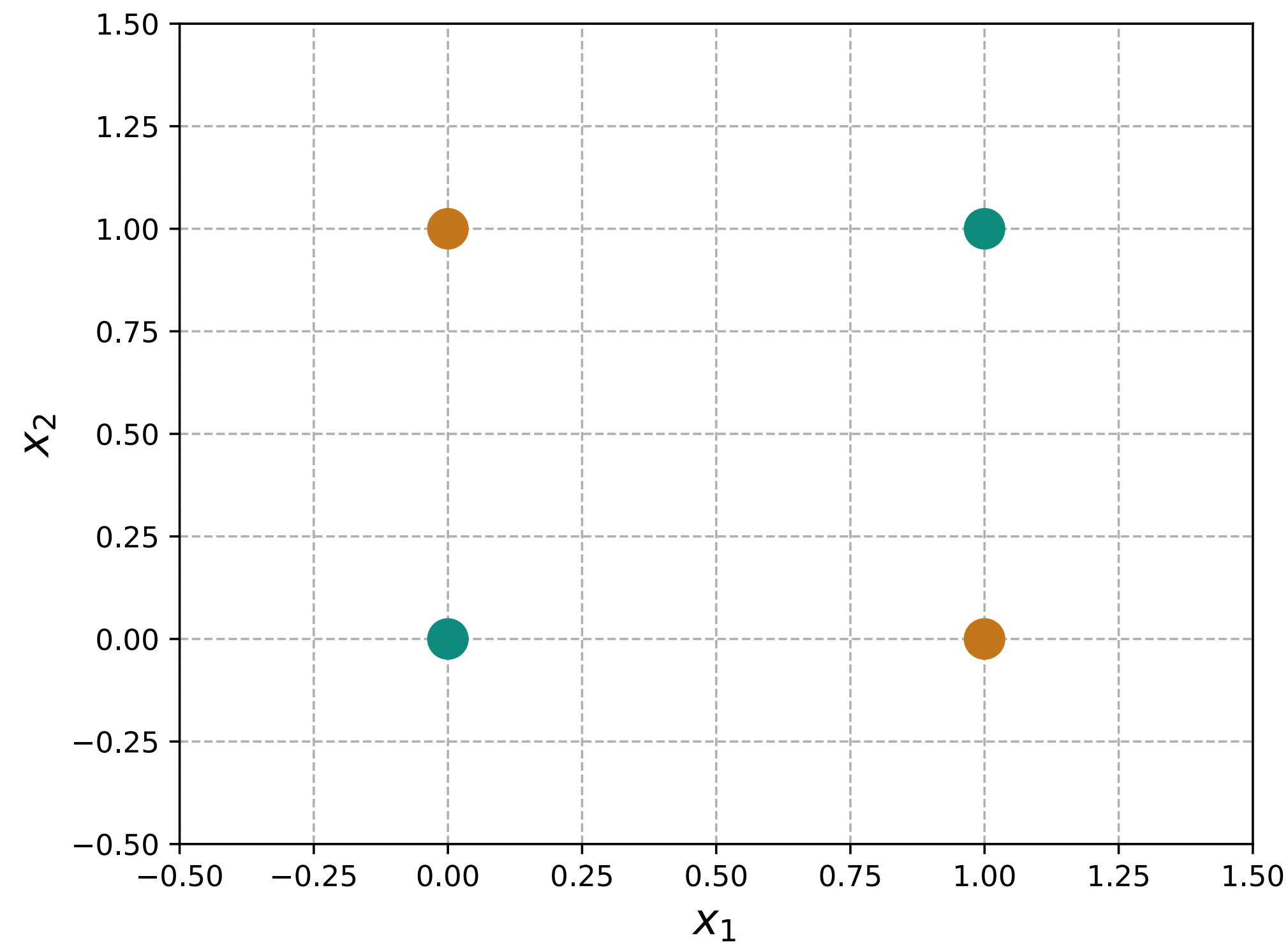
Can you cut this with a single line?



What Is a Neural Network Made Of?

The Unit: Perceptron

Can you cut this with a single line?

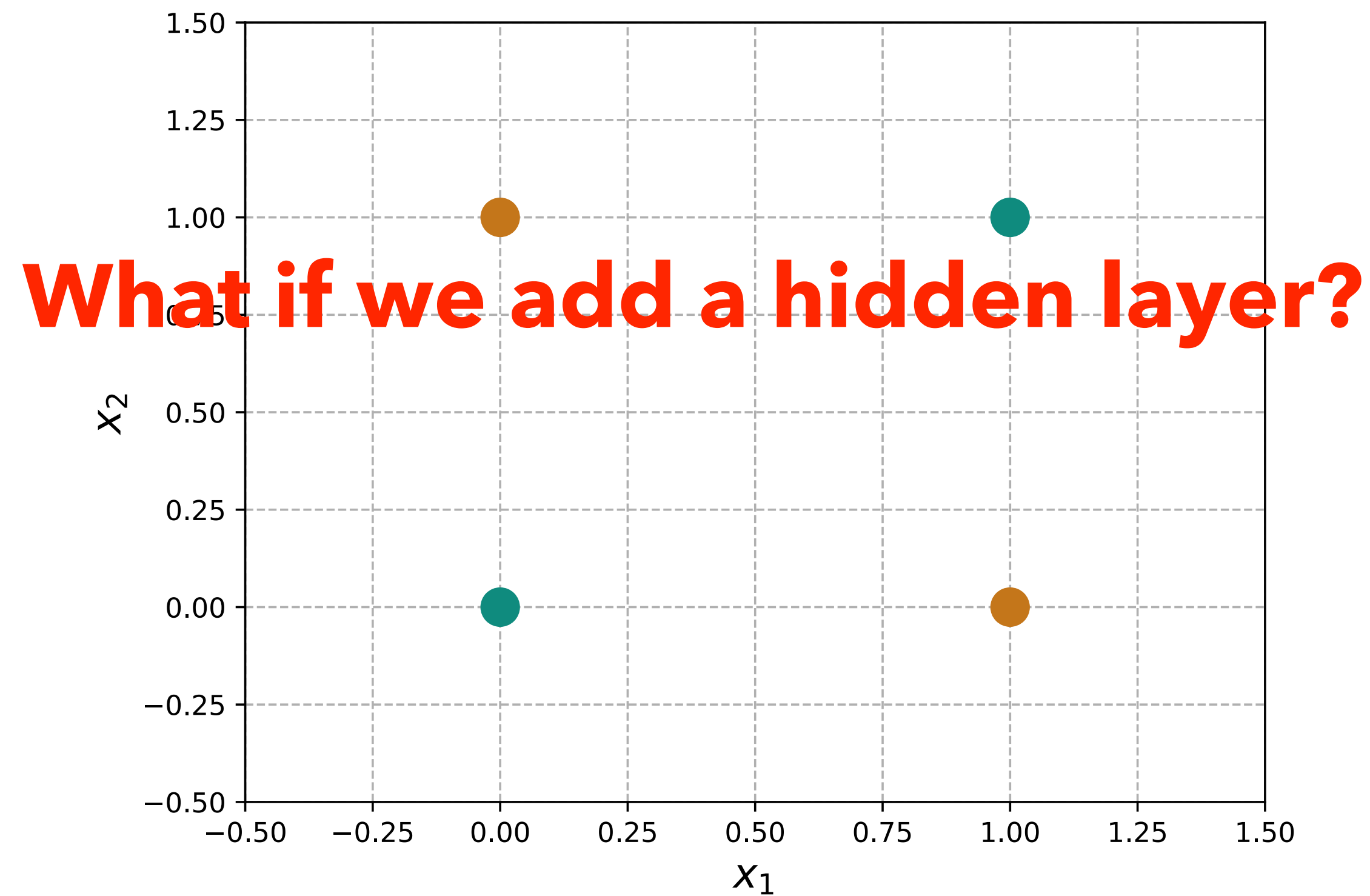


**No single line separates these four points.
A perceptron cannot represent XOR.**

What Is a Neural Network Made Of?

The Unit: Perceptron

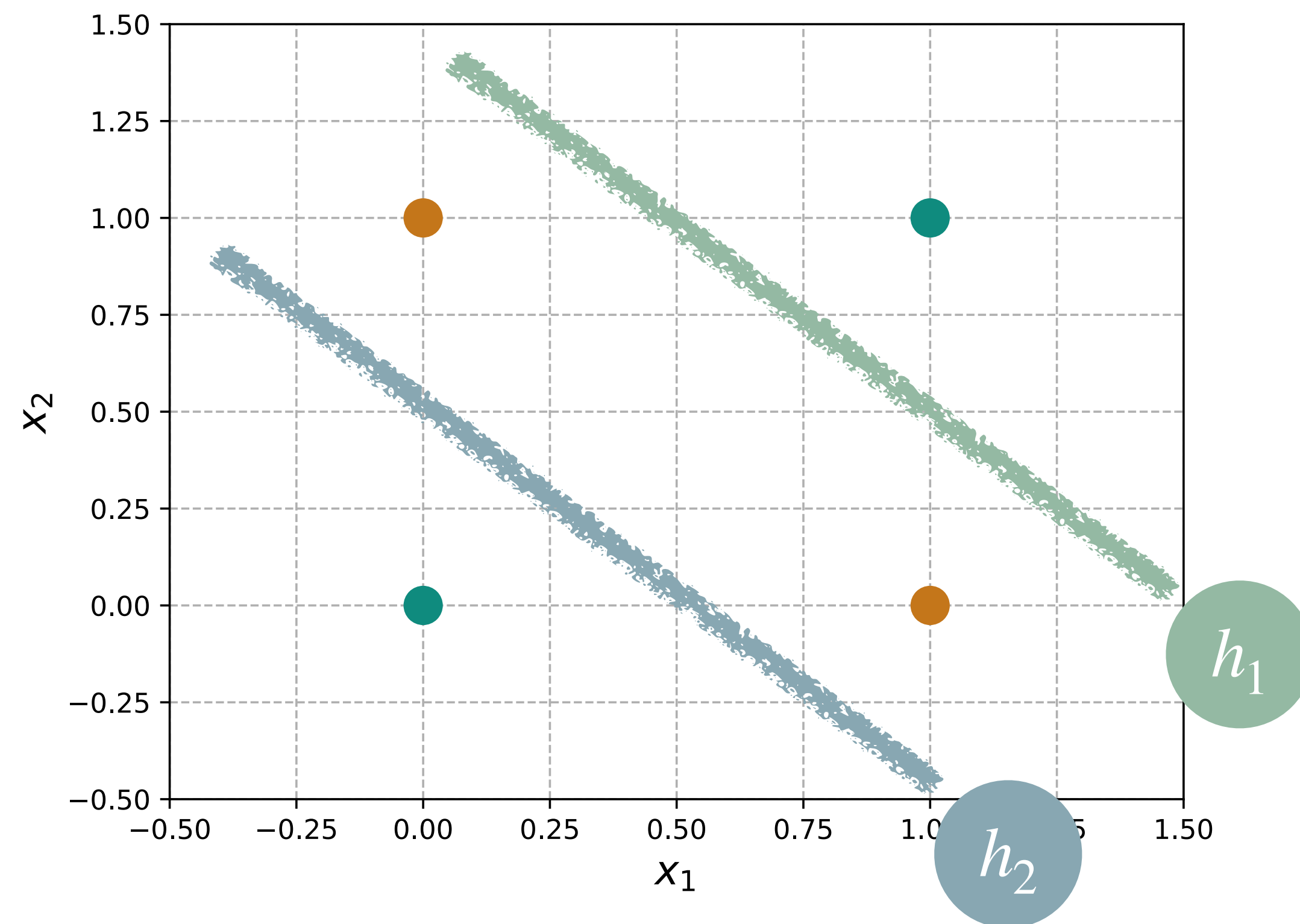
Can you cut this with a single line?



**No single line separates these four points.
A perceptron cannot represent XOR.**

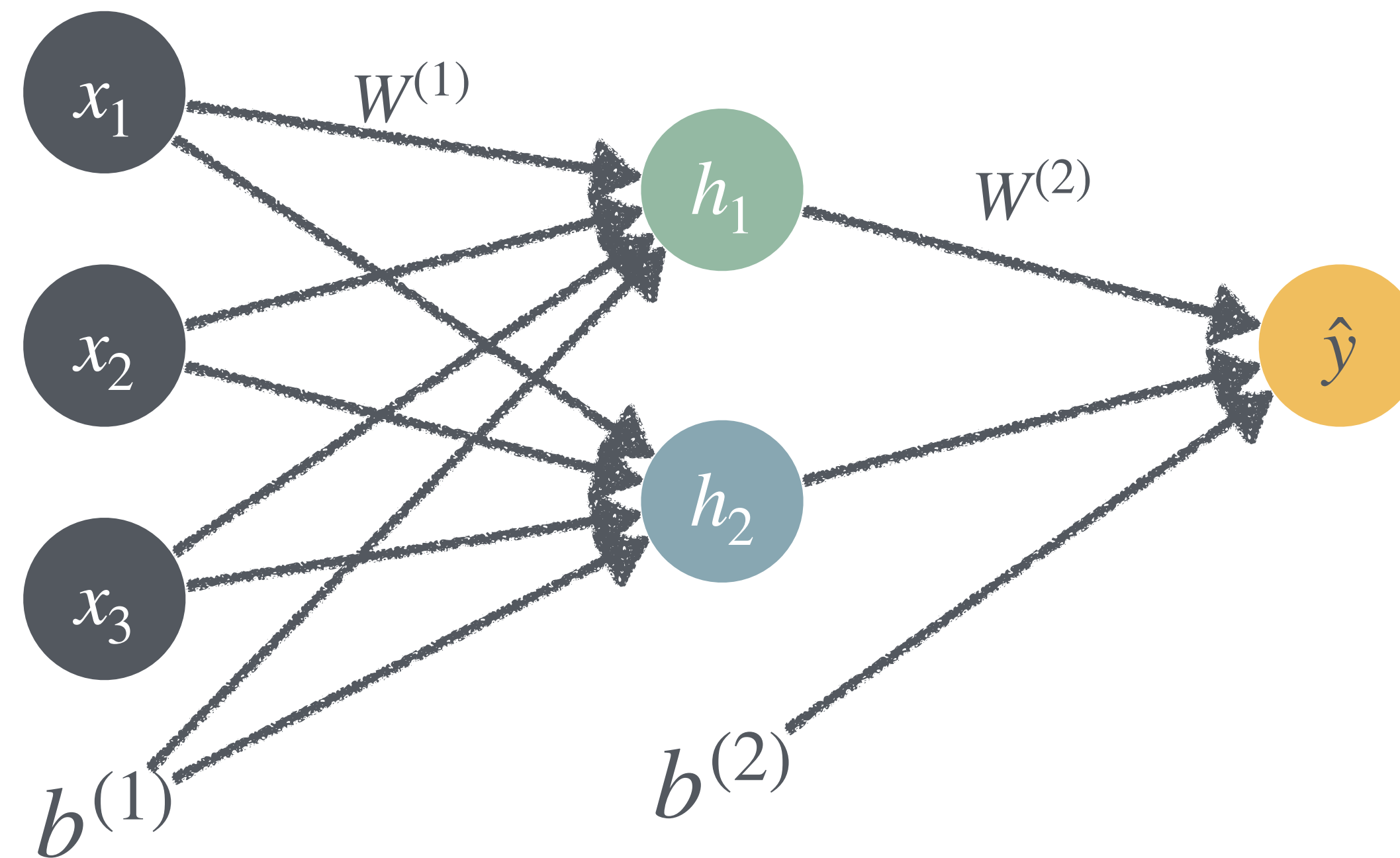
What Is a Neural Network Made Of?

Multilayer Perceptron (MLP)



What Is a Neural Network Made Of?

Multilayer Perceptron (MLP)



$$\hat{y} = g\left(\sum_k W_k^{(2)} \sigma\left(\sum_i W_{ki}^{(1)} x_i + b_k^{(1)}\right) + b^{(2)}\right)$$

σ : hidden layer activation
 g : output layer activation
They need not be the same.

Nonlinearity: Core of NN

Why Stacking Is Not Enough

- MLP worked because H is nonlinear. But H has no gradient (which we will learn is important), so we will replace it. Can we replace it with anything? Try the simplest: $\sigma = I$

$$a^{(1)} = W^{(1)}x + b^{(1)}$$

$$a^{(2)} = W^{(2)}a^{(1)} + b^{(2)} = W^{(2)}W^{(1)}x + (W^{(2)}b^{(1)} + b^{(2)})$$

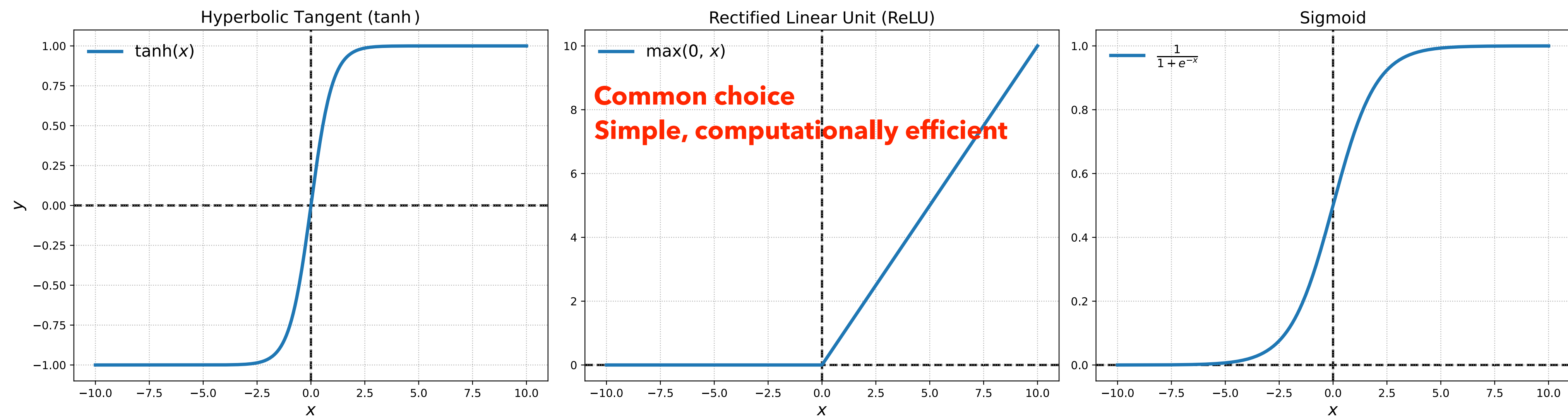
$$\equiv W'x + b'$$

- A composition of affine transformations is still an affine transformation.
- Nonlinear activations generally prevent the network from collapsing into a single affine transformation.
- A hundred layers, still one hyperplane.

Nonlinearity: Core of NN

Activation Function

- The H has zero gradient almost everywhere, making it unsuitable for standard gradient-based backpropagation.
- We use activations that are differentiable almost everywhere, such as ReLU or sigmoid.



There are other activation functions nowadays: Leaky ReLU, GELU, ...

NN as a Function Approximator

Universal Approximation Theorem

A feed-forward network with a single hidden layer, a non-polynomial activation, and enough units can approximate any continuous function on a compact set to any accuracy.

$$\forall \varepsilon > 0 \quad \exists N \in \mathbb{N}, \exists \theta : \sup_{\vec{x} \in K} |f(\vec{x}) - \phi_{N,\theta}(\vec{x})| < \varepsilon$$

- Cybenko, 1989: Sigmoidal activations
- Hornik, 1991: Bounded, nonconstant activations
- Leshno et al., 1993: Essentially any non-polynomial activation

NN as a Function Approximator

Universal Approximation Theorem

A feed-forward network with a single hidden layer, a non-polynomial activation, and enough units can approximate any continuous function on a compact set to any accuracy.

$$\forall \varepsilon > 0 \exists N \in \mathbb{N}, \exists \theta : \sup_{\vec{x} \in K} |f(\vec{x}) - \phi_{N,\theta}(\vec{x})| < \varepsilon$$

N : number of neurons θ : set of parameters

- Cybenko, 1989: Sigmoidal activations
- Hornik, 1991: Bounded, nonconstant activations
- Leshno et al., 1993: Essentially any non-polynomial activation

What It Does Not Say

Limitation of Universal Approximation Theorem

- For any ε there exists some N – but the theorem gives no bound on how large it must be.
- It doesn't say how to find the parameters. A good approximation exists somewhere in the parameter space; whether we can reach it is a separate question.

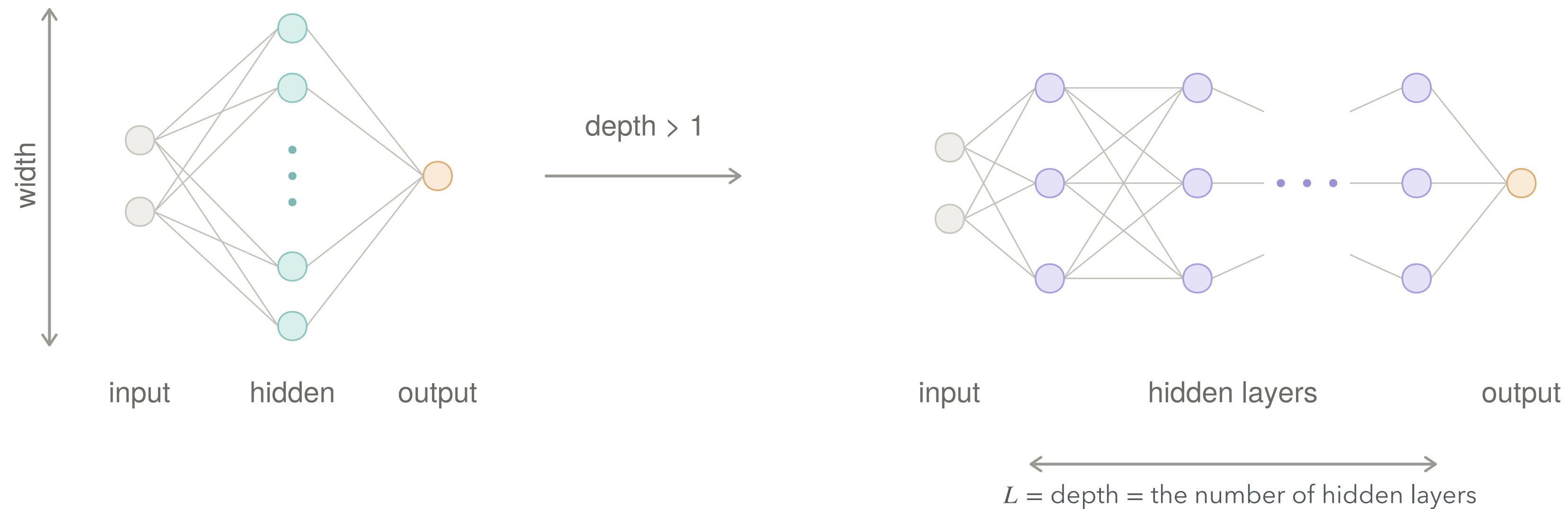
Deep Neural Network

Stacking Up

A deep neural network is an MLP with more hidden layers.

Shallow

Deep

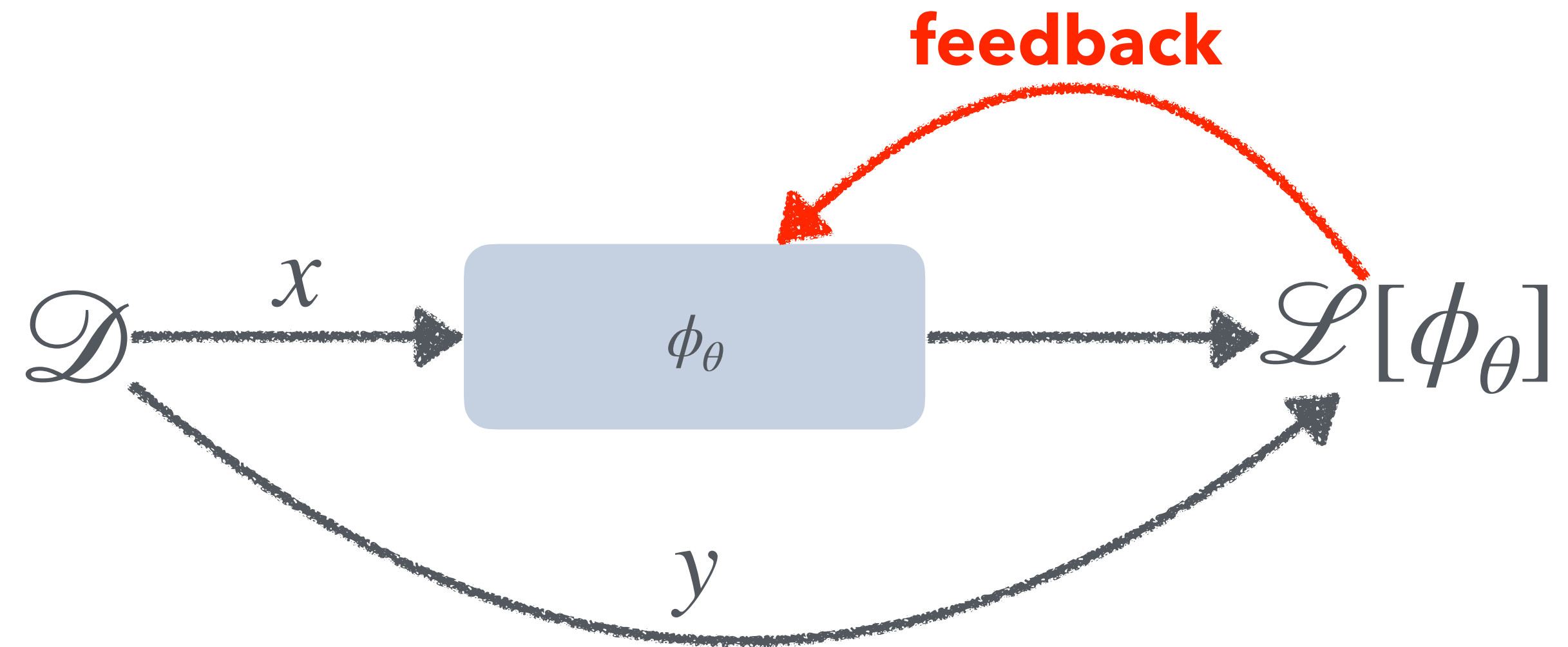


$$\hat{y} = (f^{(\text{out})} \circ f^{(L)} \circ f^{(L-1)} \circ \dots \circ f^{(1)})(\vec{x})$$

$$f^{(\ell)}(\vec{a}) = \sigma(W^{(\ell)}\vec{a} + \vec{b}^{(\ell)}), \quad f^{(\text{out})}(\vec{a}) = g(W^{(\text{out})}\vec{a} + \vec{b}^{(\text{out})})$$

Training Neural Network

Big Picture



- Data $\mathcal{D} = \{(x_i, y_i) \in \mathbb{R}^d \times \mathbb{R}^k\}_{i=1}^{|\mathcal{D}|}$
- Network $\phi_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^k$
- Parameters θ
- Loss function $\mathcal{L}[\phi_\theta] = \frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} \ell(\phi_\theta(x_i), y_i)$

Preparing Data

Three Splits, Three Jobs

- **Training set** – the only data the optimizer ever sees. The parameters are fitted to this, and to nothing else.
- **Validation set** – never trained on, only watched. It is how we choose the things that the network cannot choose for us: how long to train, how big to make the network (these choices are called **hyperparameters**)
- **Test set** – touched once, at the very end. Every time we look at it and then change something in response, it stops being an honest estimate of performance on unseen data.

Preparing Data

Good Data Makes a Good Network

- Input features should share a common scale.
- Different scales can make optimization poorly conditioned, slow, or unstable.
- Preprocessing is computed on the **training set** only, then applied unchanged to validation and test.
- Commonly used

A. Standardization: $x \rightarrow \frac{x - \mu}{\sigma}$ – mean 0, standard deviation 1

B. Min-max normalization: $x \rightarrow \frac{x - x_{\min}}{x_{\max} - x_{\min}}$ – maps the range to [0,1]

Initialization

Where Training Starts

- Weights cannot all start at the same value.
- If they did, every unit in a layer would compute the same thing and receive the same gradient – identical forever, however many of them there are.
- So we start randomly

Xavier for **tanh/sigmoid**:

$$W \sim \mathcal{N}(0, \frac{2}{n_{\text{in}} + n_{\text{out}}})$$

He for **ReLU**:

$$W \sim \mathcal{N}(0, \frac{2}{n_{\text{in}}})$$

Epoch and Batch

Efficient Ways to Train NN

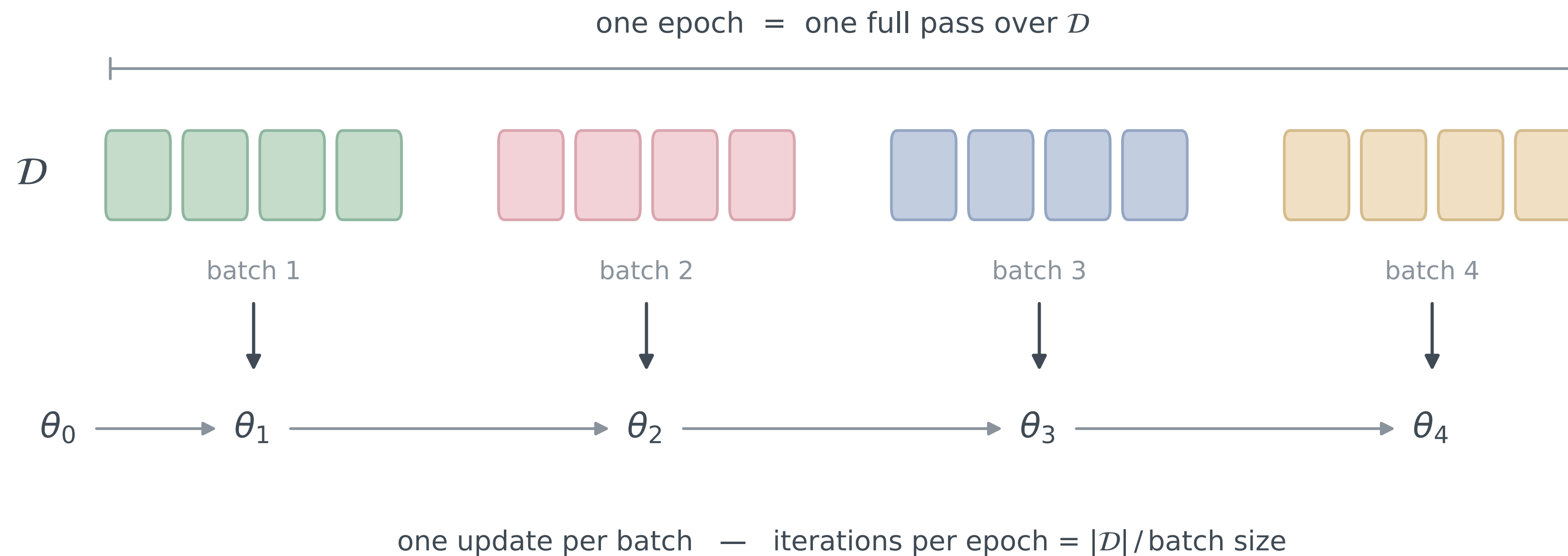
- Training is iterative – the same training data is used over and over, so it has to be fed efficiently.
- One **epoch** is one full pass through the training set. Within a single epoch, the parameters are updated many times.
- Instead of using all of $\mathcal{D}$ for every update, we split it into **(mini) batches** and take one step per batch. The batch average is a cheap estimate of the full loss: accurate enough to move in the right direction, small enough to compute quickly. Batch size is usually a power of two.

$$\mathcal{D}_{\text{train}} = \mathcal{B}_1 \cup \mathcal{B}_2 \cup \dots \cup \mathcal{B}_{N_{\mathcal{B}}}$$

- The number of epochs and the batch size are not learned during training – we choose them by hand. These are called **hyperparameters**.

Epoch and Batch

Efficient Ways to Train NN



Loss Function

Heart of Optimization

- The loss turns “how wrong is the network” into a single number.
- $\ell(\hat{y}, y)$ scores one prediction; $\mathcal{L}$ averages it over the dataset. Training means finding the θ that makes $\mathcal{L}$ small

$$\theta^* = \arg \min_{\theta} \mathcal{L}[\phi_{\theta}]$$

- A usable loss must be small when $\hat{y} \approx y$ and **differentiable** in θ – otherwise there is no gradient to follow.
- Two standard choices

A. Regression – mean squared error (MSE): $\ell = \frac{1}{2}(\hat{y} - y)^2$

B. Classification – cross-entropy: $\ell = - \sum_k y_k \log \hat{y}_k$ with $\hat{y} = \text{softmax}(\phi_{\theta}(x))$

Cross-entropy

- One-hot encoding – the label becomes a vector.

Class 2 out of 3: $y = (0,1,0)^T$

- Softmax – raw scores become probabilities that sum to unity.

$$\hat{y}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \quad z = (1,3,0) \rightarrow \hat{y} = (0.114, 0.844, 0.042)$$

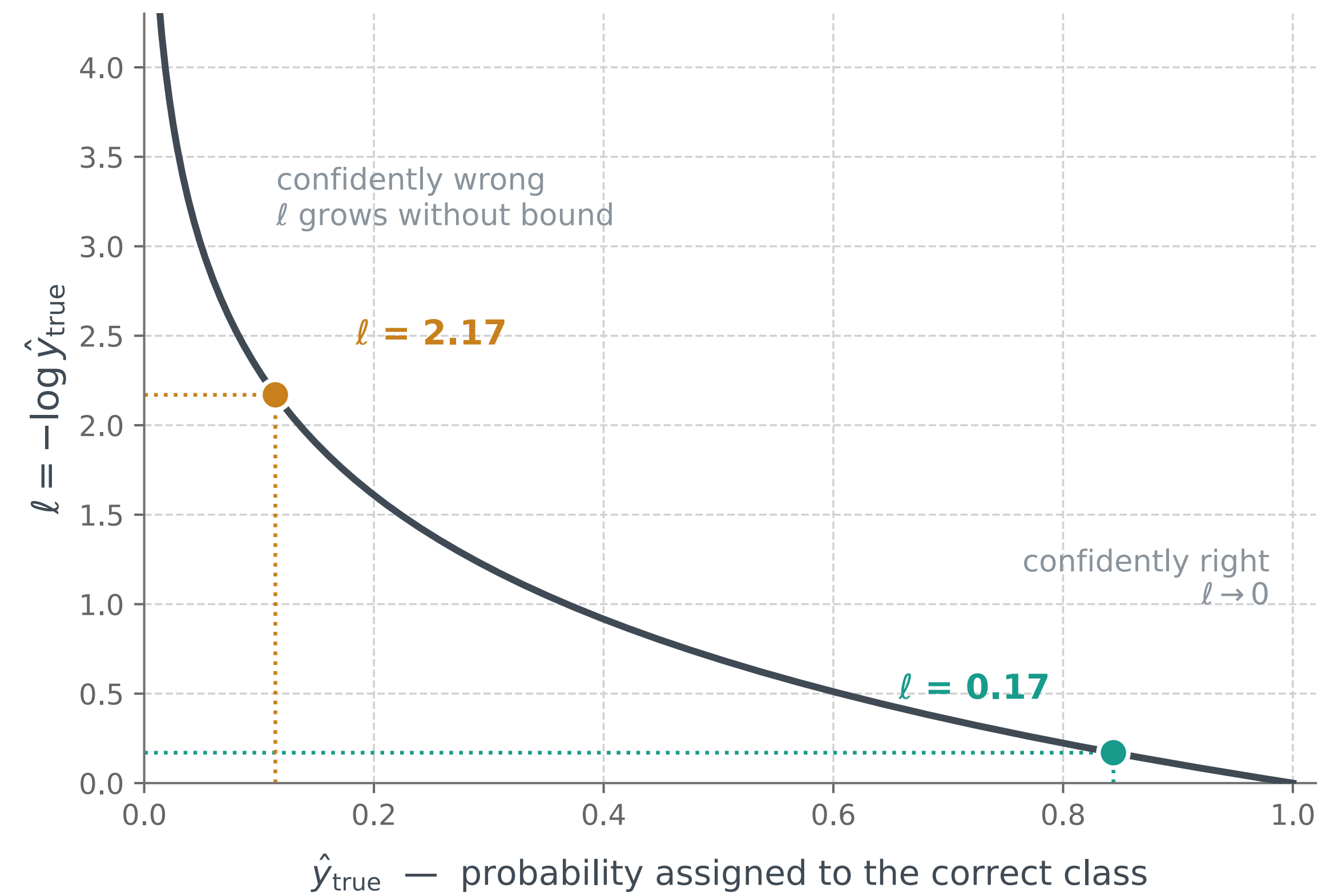
- Cross-entropy – because y is one-hot, every term but one vanishes.

$$\ell = - \sum_k y_k \log \hat{y}_k = - \log \hat{y}_{\text{true}}$$

- So the loss asks a single question: what probability did the network give to the correct answer?

$$\hat{y}_{\text{true}} = 0.844 \rightarrow \ell = 0.170, \quad \hat{y}_{\text{true}} = 0.114 \rightarrow \ell = 2.17$$

Cross-entropy



Cross-entropy

It Is Just Maximum Likelihood

- Each data point gets a probability from the network; assuming they are i.i.d., the dataset has a likelihood

$$L(\theta) = \prod_i P(y_i | x_i) = \prod_i \prod_k (\hat{y}_{ik})^{y_{ik}}$$

$$\longrightarrow \log L(\theta) = \sum_i \sum_k y_{ik} \log \hat{y}_{ik}$$

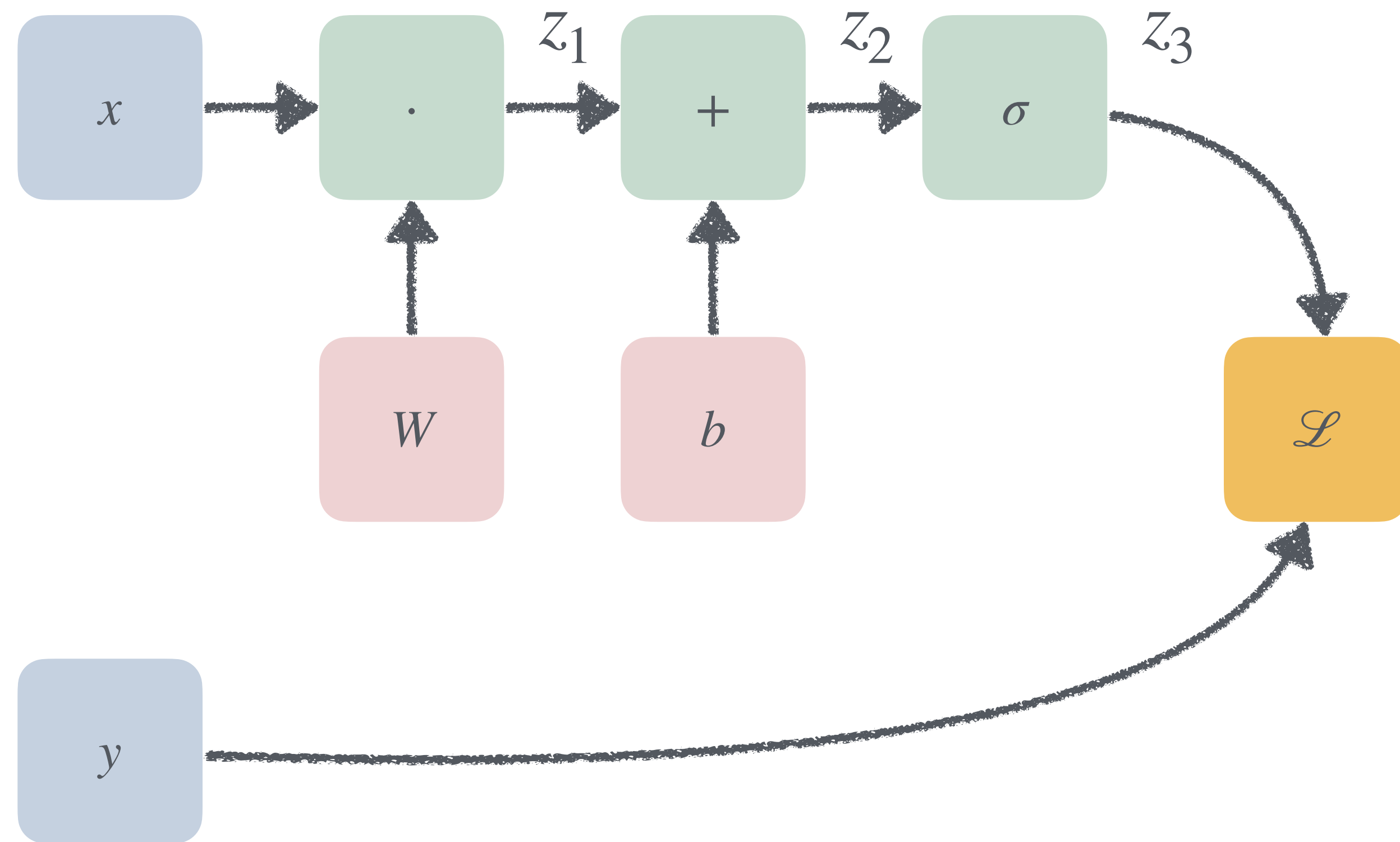
- Flip the sign and divide by $|D|$ – this is exactly the loss we already wrote

$$\mathcal{L}[\phi_\theta] = -\frac{1}{|D|} \sum_i \sum_k y_{ik} \log \hat{y}_{ik} = -\frac{1}{|D|} \log L(\theta)$$

- Minimizing cross-entropy is maximum likelihood estimation – the same principle behind many familiar fitting methods.

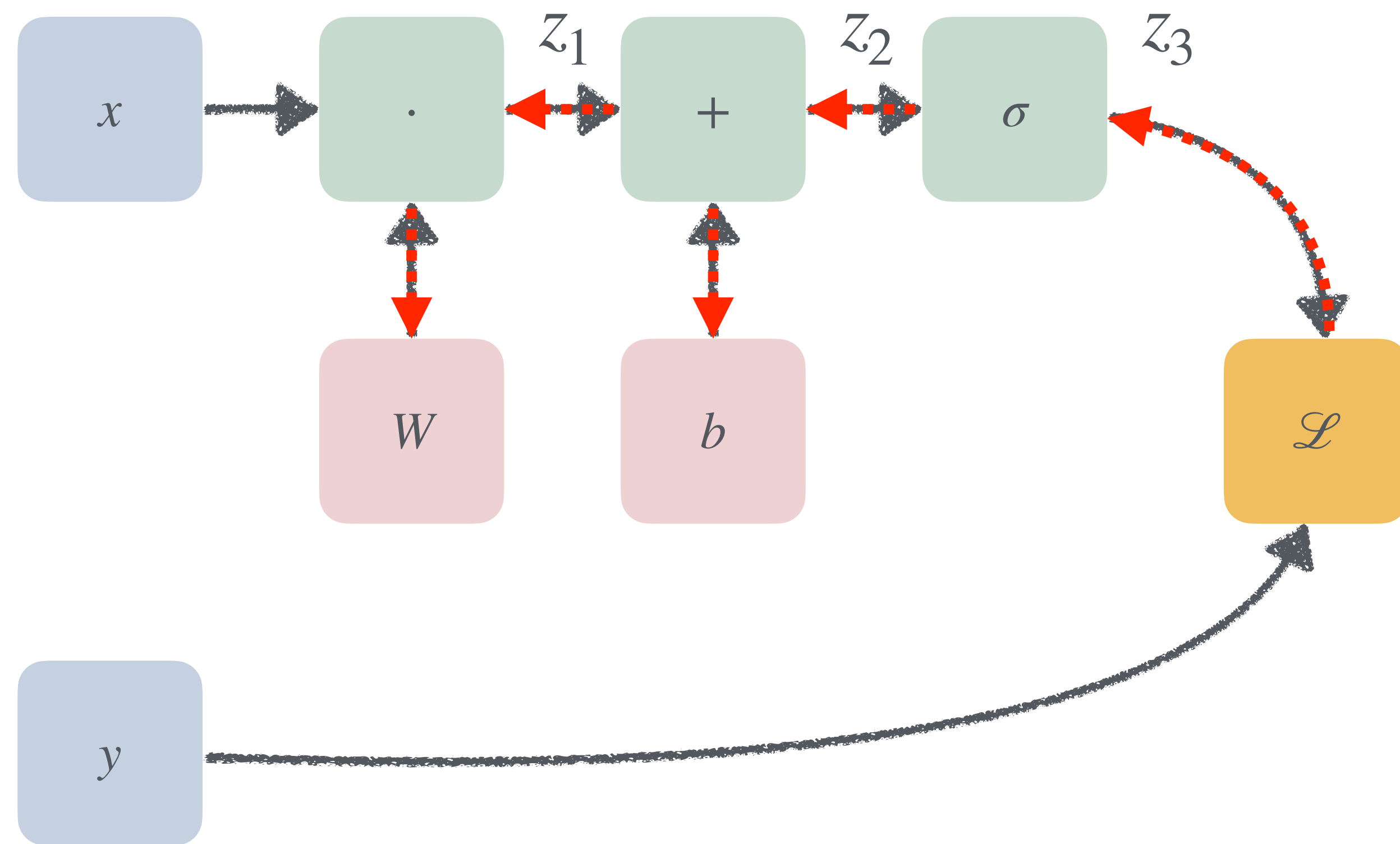
Backpropagation

Forward: compute and remember



Backpropagation

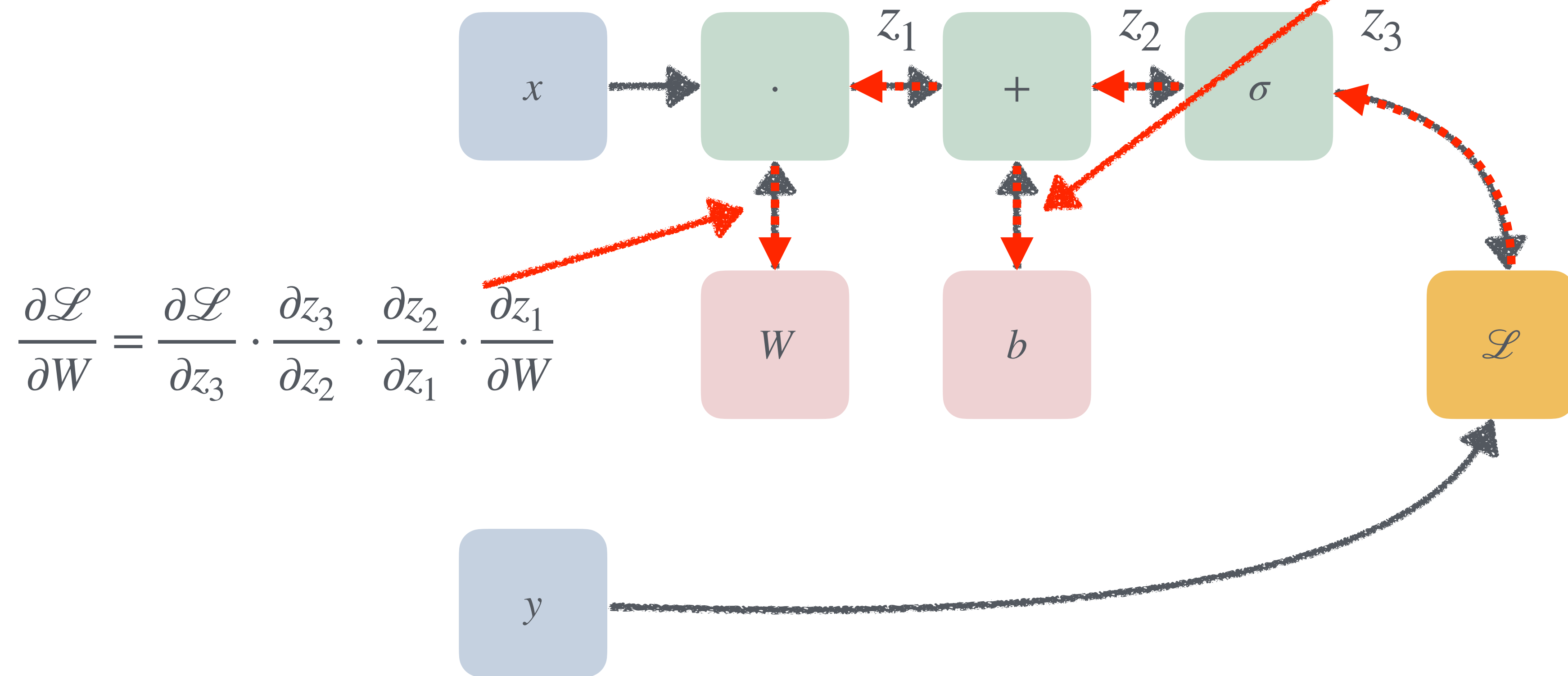
Backward: calculating the gradient



Backpropagation

Backward: calculating the gradient

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial z_3} \cdot \frac{\partial z_3}{\partial z_2} \cdot \frac{\partial z_2}{\partial b}$$



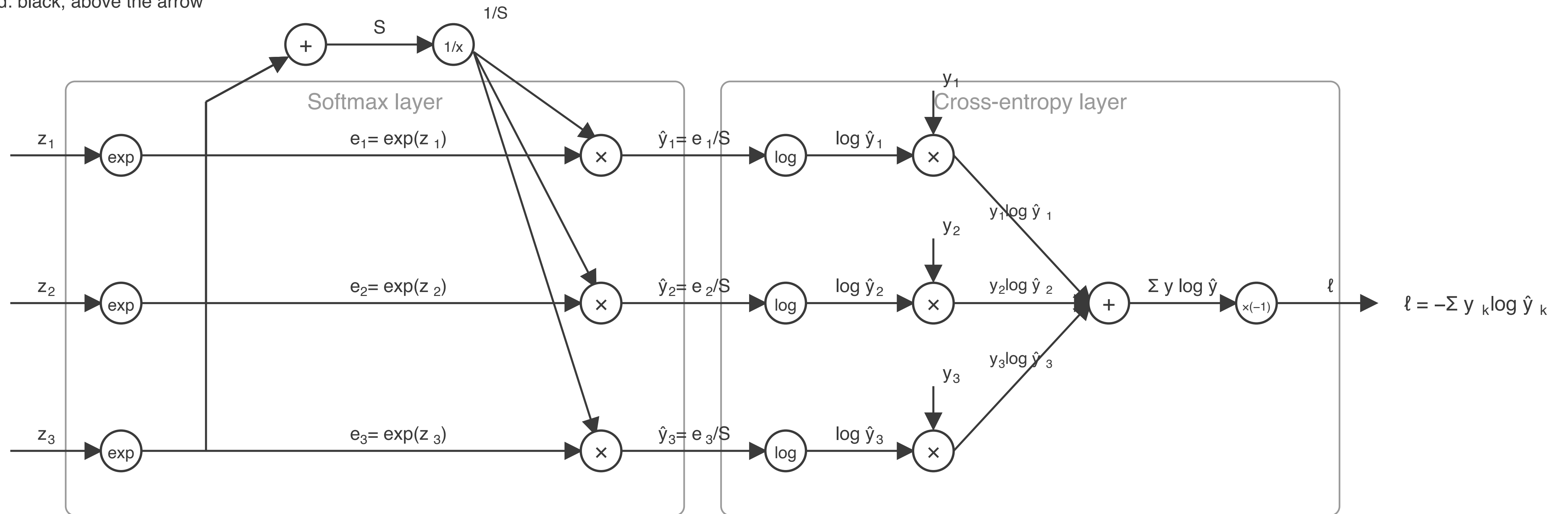
Backpropagation = the chain rule, applied from the loss backwards, one node at a time

Backpropagation

Computational Graph

Let's do this by hand!

forward: black, above the arrow



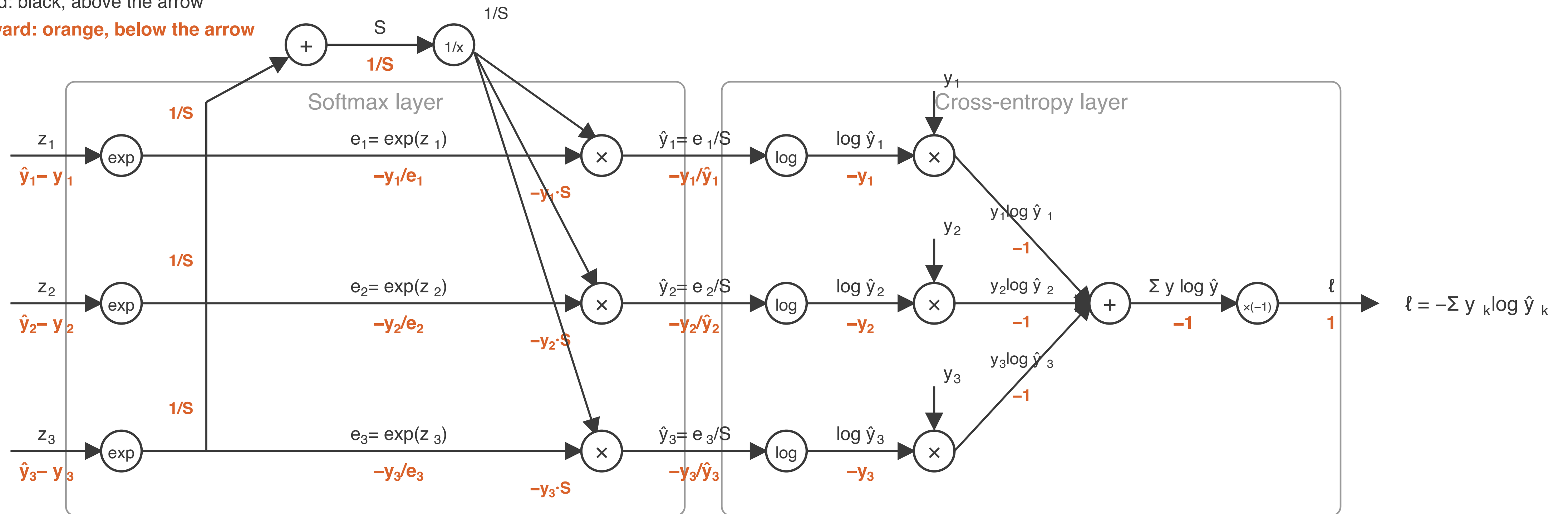
Backpropagation

Computational Graph

Did you get the same answer? That result is no coincidence.

forward: black, above the arrow

backward: orange, below the arrow



backward: multiply the local derivative at each node, from ℓ back to z

No Coincidence

Output and Loss Come in Pairs

Output function	Loss	$P(y x)$	$\frac{\partial \ell}{\partial z}$
Identity	MSE	Gaussian	$\hat{y} - y$
Sigmoid	Binary cross-entropy	Bernoulli	
Softmax	Categorical cross-entropy	Categorical	

- Each loss is the **negative log-likelihood** of the distribution that its output function produces.
- The logarithm undoes the exponential, so the gradient collapses to **prediction minus truth**.
- What we derived for softmax is one instance of a general pattern, not a lucky cancellation.

Stochastic Gradient Descent (SGD)

Toward the Optimal Parameters

- Backpropagation gives $\nabla_{\theta}\mathcal{L}$ – one number for every parameter. Optimization is what the network does with it.
- The gradient points uphill, so we step the other way

$$\theta \leftarrow \theta - \eta \nabla_{\theta}\mathcal{L}$$

- η is the **learning rate**: how far to step. This is another **hyperparameter** we have to choose.
- Parameters are updated per batch. Each batch gives a cheap, noisy estimate of the full-data gradient – and the noise sometimes helps to escape from unwanted local minima

$$\hat{\nabla}_{\theta}\mathcal{L} = \frac{1}{|\mathcal{B}|} \sum_i^{\mathcal{B}} \nabla_{\theta}\ell_i, \quad \mathbb{E}[\hat{\nabla}_{\theta}\mathcal{L}] = \nabla_{\theta}\mathcal{L}$$

Momentum

Remembering the Last Step

- SGD treats every step as if it were the first.
- Momentum keeps a running average of past gradients and moves along that instead

$$m \leftarrow \beta_1 m + (1 - \beta_1) \nabla_{\theta} \mathcal{L}, \quad \theta \leftarrow \theta - \eta m$$

- Newly introduced m is called momentum and β_1 is another hyperparameter that controls how much of the past is kept.

Adaptive Learning Rates

AdaGrad $\rightarrow$ RMSProp

- The learning rate matters. Too small and training crawls; too large and it diverges.
- Learning rate decay is one approach: start large, shrink step by step.
- **AdaGrad** goes further and **adapts** it per parameter

$$v \leftarrow v + (\nabla_{\theta} \mathcal{L})^2, \quad \theta \leftarrow \theta - \eta \frac{1}{\sqrt{v} + \varepsilon} \nabla_{\theta} \mathcal{L}$$

- No new hyperparameter to tune – ε is just a small constant that avoids dividing by zero.
- **But v only grows.** The effective step decays toward zero, and training can stall before it should.

Adaptive Learning Rates

AdaGrad $\rightarrow$ RMSProp

- **RMSProp** fixes it with a single change – replace the sum with a moving average

$$v \leftarrow \beta_2 v + (1 - \beta_2) (\nabla_{\theta} \mathcal{L})^2$$

- The past is forgotten, so v stops growing and the step size settles instead of vanishing.
- β_2 is the **decay rate** of the average – how much of the past is kept.

Adaptive Moment Estimation (Adam)

Using Both Moments

- Momentum provided the direction, and RMSProp provided the per-parameter step size.
- Adam keeps both

$$m \leftarrow \beta_1 m + (1 - \beta_1) \nabla_{\theta} \mathcal{L}$$

$$v \leftarrow \beta_2 v + (1 - \beta_2) (\nabla_{\theta} \mathcal{L})^2$$

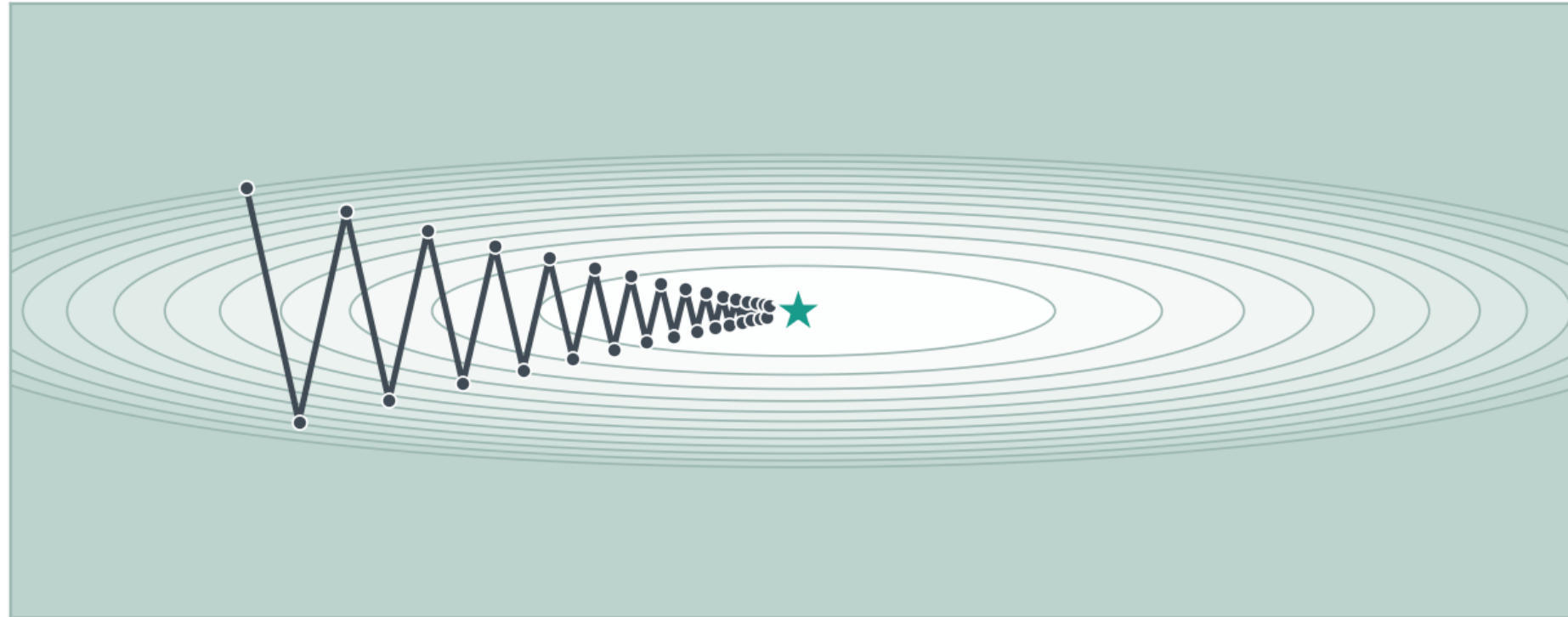
$$\hat{m} = \frac{m}{1 - (\beta_1)^t}, \quad \hat{v} = \frac{v}{1 - (\beta_2)^t} \quad t \text{ is the update counter}$$

$$\theta \leftarrow \theta - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$

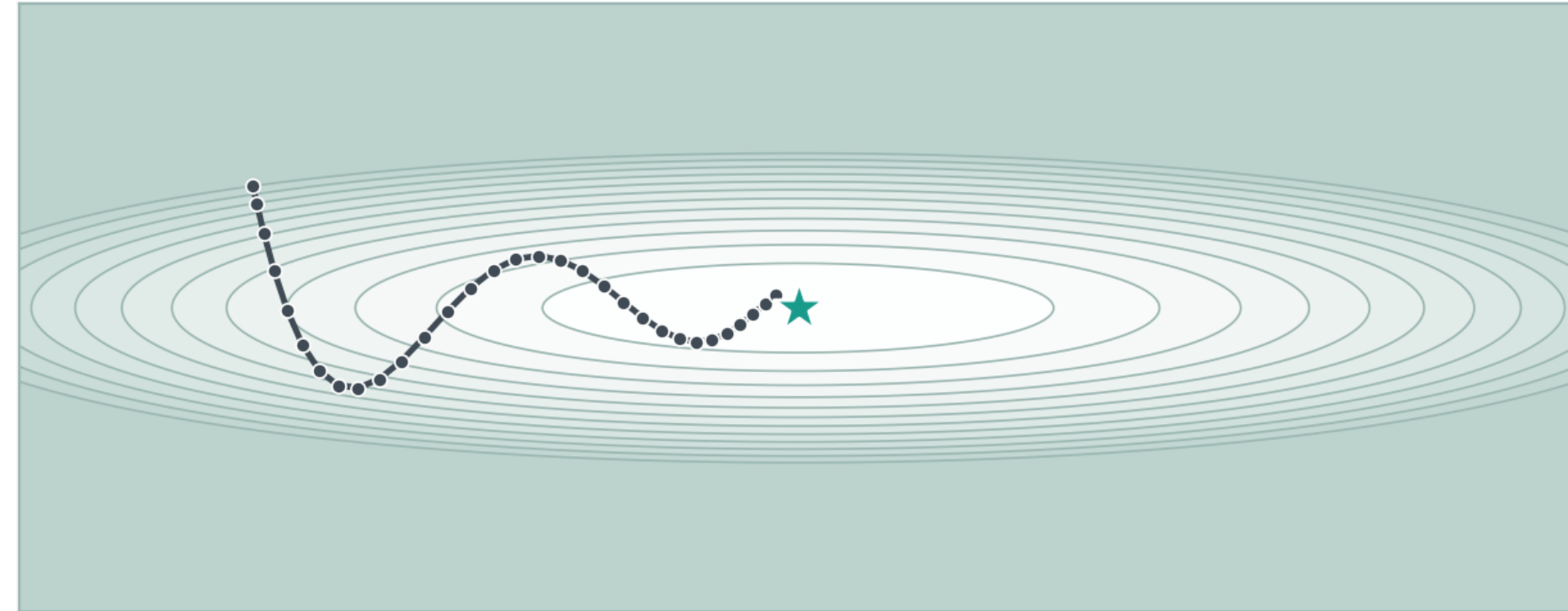
- m and v both start at zero, so early on, they are biased toward it. Dividing by $(1 - (\beta_{1,2})^t)$ undoes exactly that.

Which is Best?

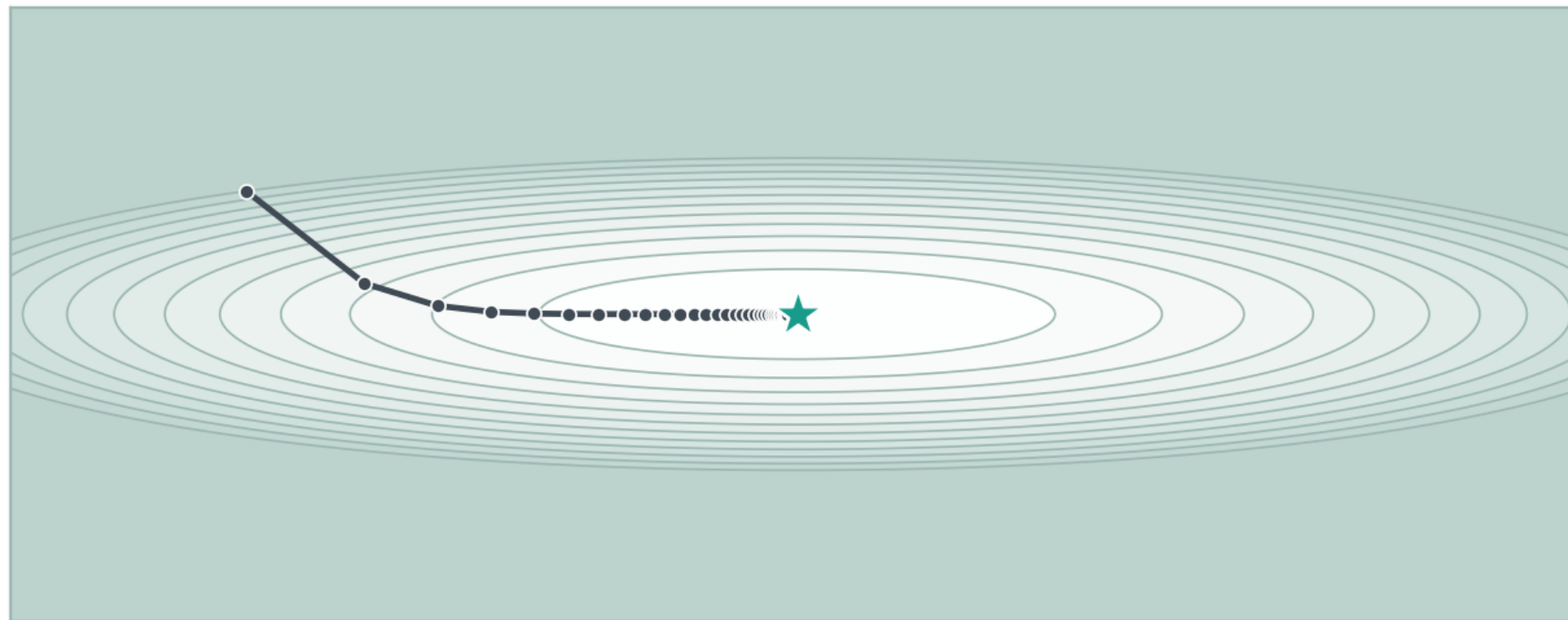
SGD



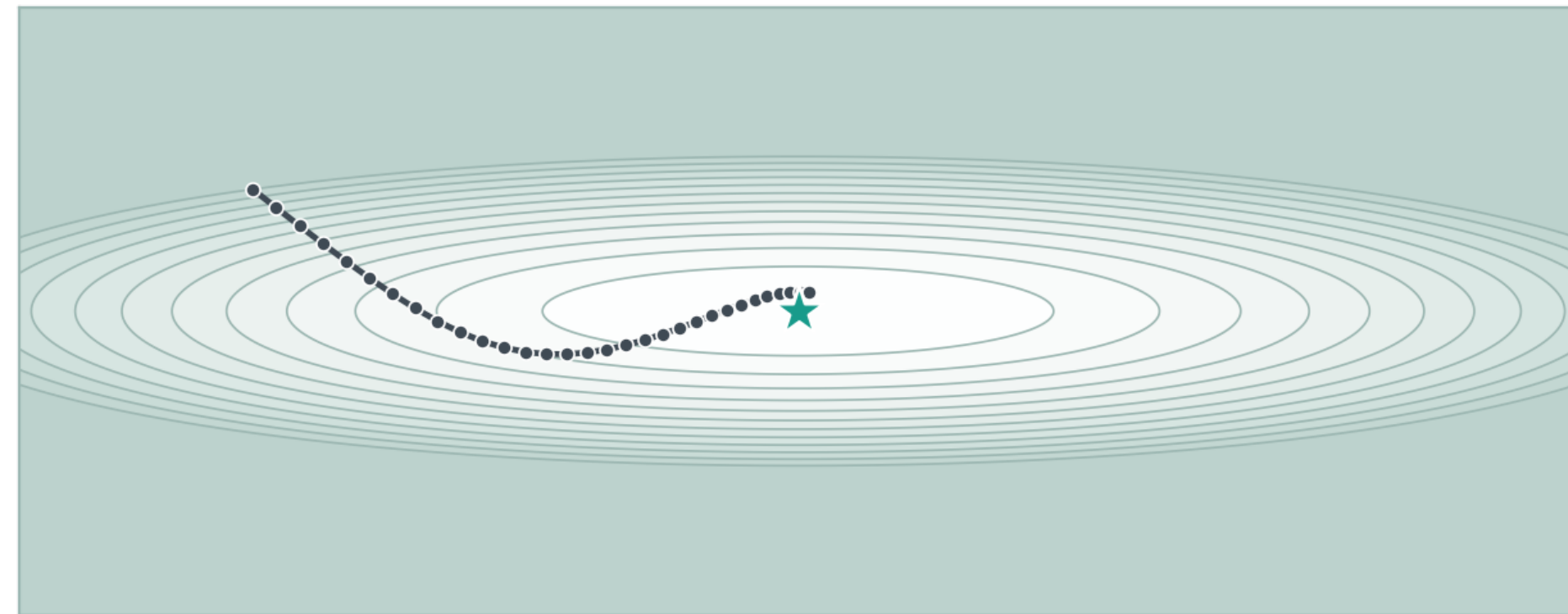
Momentum



AdaGrad



Adam

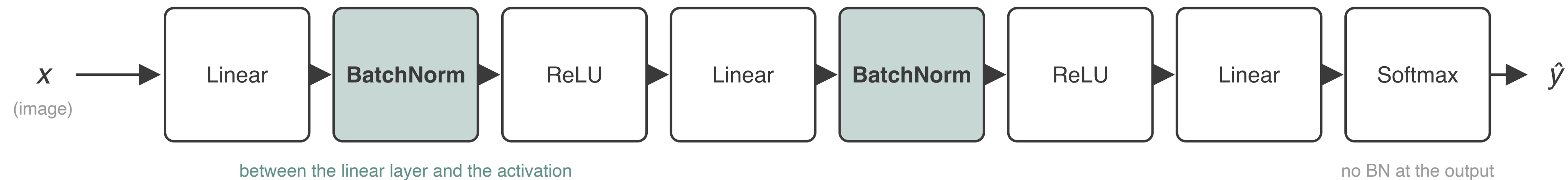


$f(x, y) = x^2/20 + y^2$, same start, 30 steps

AdaGrad looks best here. But the ranking depends on the problem; it changes with the hyperparameters, and there is still **no method that is best everywhere**.

Batch Normalization

Normalizing the Middle Layers



- We standardized the inputs. But after a few layers, the distribution has drifted again – and it keeps drifting as θ changes during training

$$\hat{z} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad \hat{z} \leftarrow \gamma \cdot \hat{z} + \beta$$

- γ rescales and β shifts; both are learned along with the weights.
- Running averages are accumulated during training and used at test time, when there is no batch

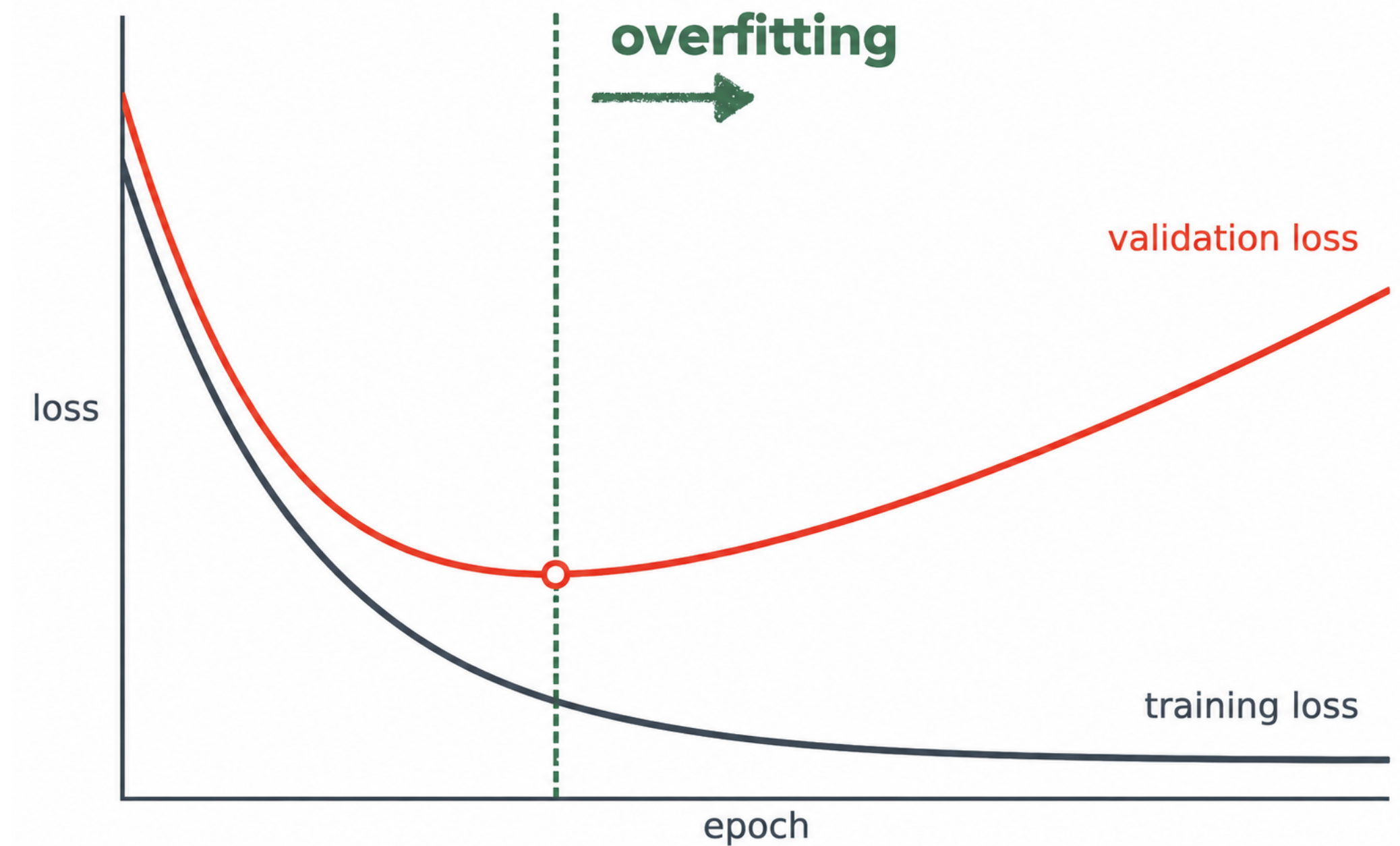
$$\bar{\mu}_B \leftarrow (1 - m)\bar{\mu}_B + m\mu_B, \quad (\bar{\sigma}_B)^2 \leftarrow (1 - m)(\bar{\sigma}_B)^2 + m(\sigma_B)^2$$

- Faster training, less sensitivity to initialization, and a mild regularizing effect.

Overfitting

The Loss We Minimize Is Not the One We Care About

- Everything so far has minimized the loss on the training set. What we actually want is low loss on data that the network has never seen.
- The validation loss is the only curve that reports on unseen data – so when it turns around, we believe it. The training loss cannot turn around: the optimizer is pushing it down by construction.
- The two curves tell the story: training loss keeps falling, validation loss turns around. That turning point is overfitting.
- Underfitting is the opposite failure – both losses stay high. The model is too small, or training stopped too early.

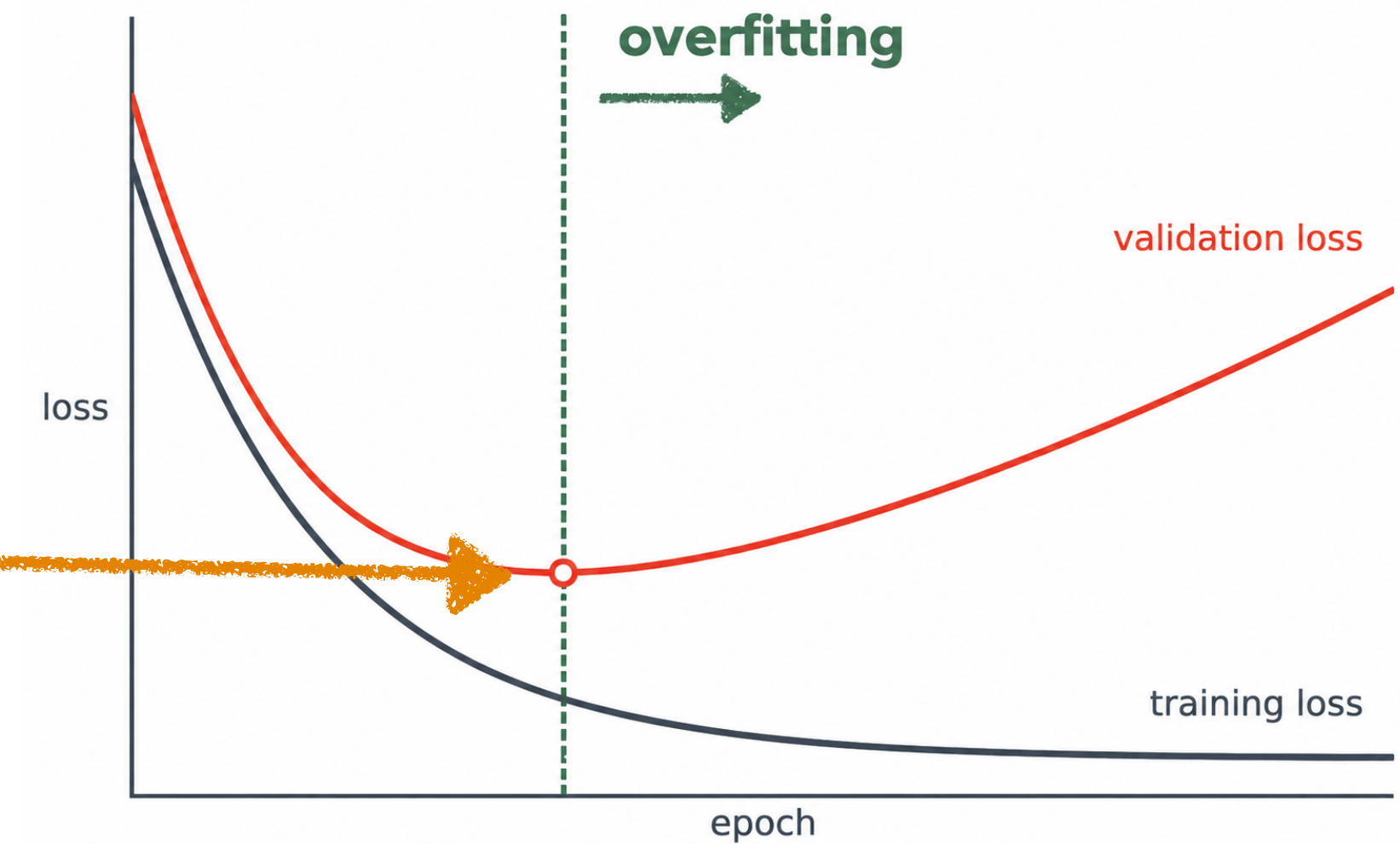


Regularization

Three Ways to Fight Overfitting

- **Early stopping** – keep the parameters from the epoch with the lowest validation loss. Costs nothing and is always worth doing.

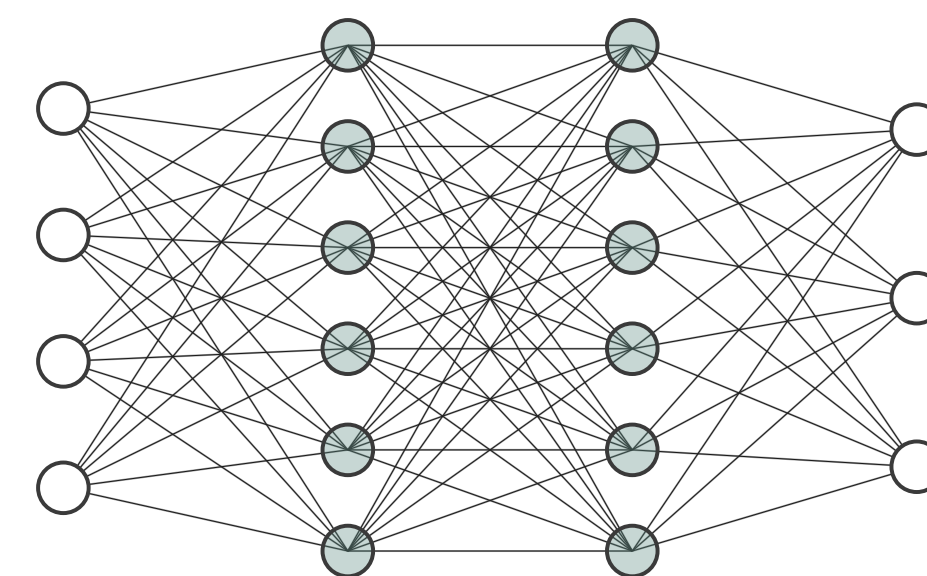
Early stopping



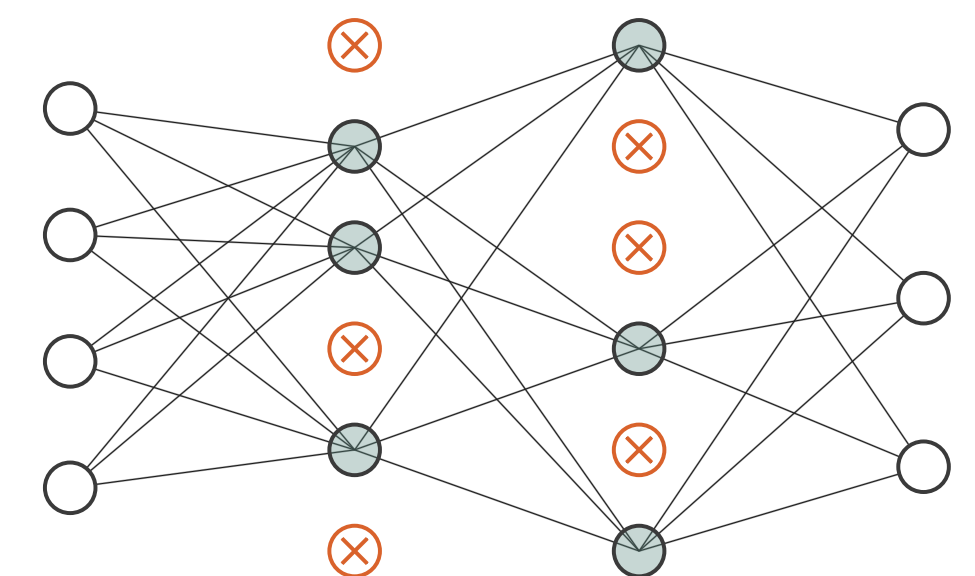
- **Weight decay** – Large weights now cost something, so the network prefers smoother solutions.

$$\mathcal{L} \leftarrow \mathcal{L} + \frac{1}{2}\lambda\|W\|^2, \quad \lambda \text{ is a hyperparameter}$$

- **Dropout** – randomly switch off a fraction of units at each step. No single unit can be relied on, so the representation spreads out. Dropout is switched off at test time; the full network is used.



full network — used at test time



dropout — a random subset is switched off at each training step

Big Picture

Every Arrow Now Has a Name

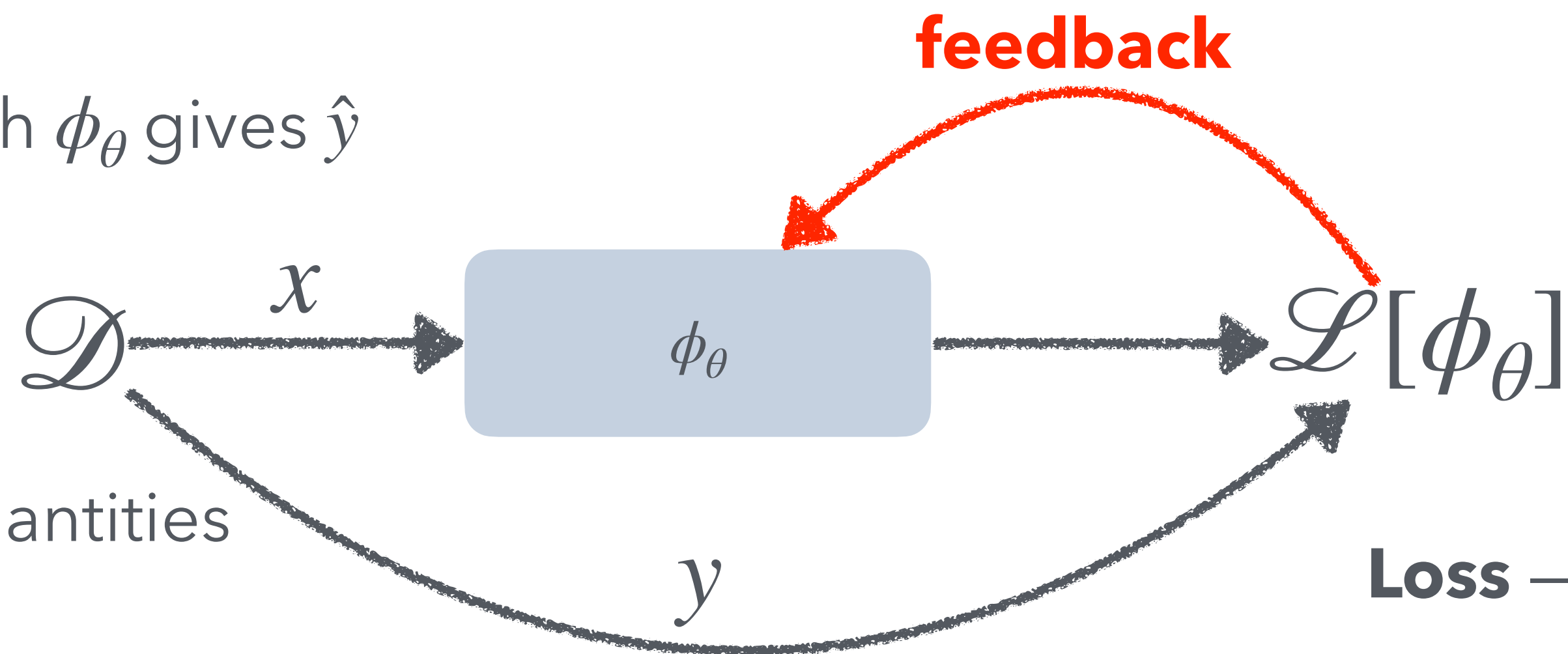
Backward – backpropagation turns $\mathcal{L}$ into $\nabla_{\theta}\mathcal{L}$

Update – $\nabla_{\theta}\mathcal{L}$ into change in θ

Forward – x through ϕ_{θ} gives $\hat{y}$

Preprocessing

– scaling the raw quantities



Loss – $\mathcal{L}$ compares $\hat{y}$ with y

As a consequence, we get ϕ_{θ^*} a network trained for the given problem

$$\theta^* = \arg \min_{\theta} \mathcal{L}[\phi_{\theta}]$$

Hands-On
MNIST

What we've covered...

- ✓ Perceptron
- ✓ MLP
- ✓ Nonlinearity
- ✓ Universal Approximation Theorem
- ✓ Loss & Optimization
- ✓ Train / Val / Test splits
- ✓ Overfitting & Regularization

Now it's time for
hands-on!

MNIST

Let's build, train, and
see it work! 😊

Theory
was the warm-up...
Now the
real fun begins! 😊



Small steps.
Big models. 😊



Same
ideas.
Bigger
data 🐣

- ☐ Load data
- ☐ Build the model (MLP)
- ☐ Train & evaluate
- ☐ See the results! ✨

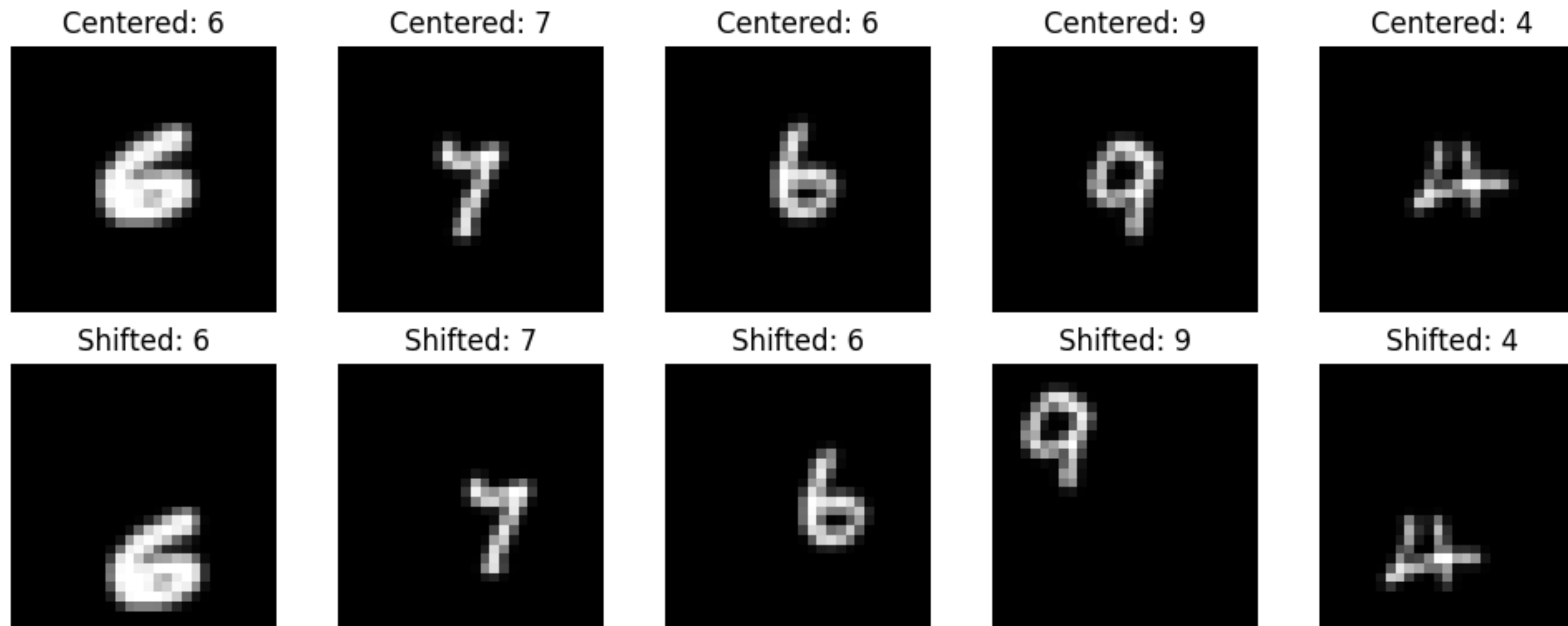


Convolutional Neural Network

With MNIST Example

The Same Digit, Moved

Our Eyes Do Not Care



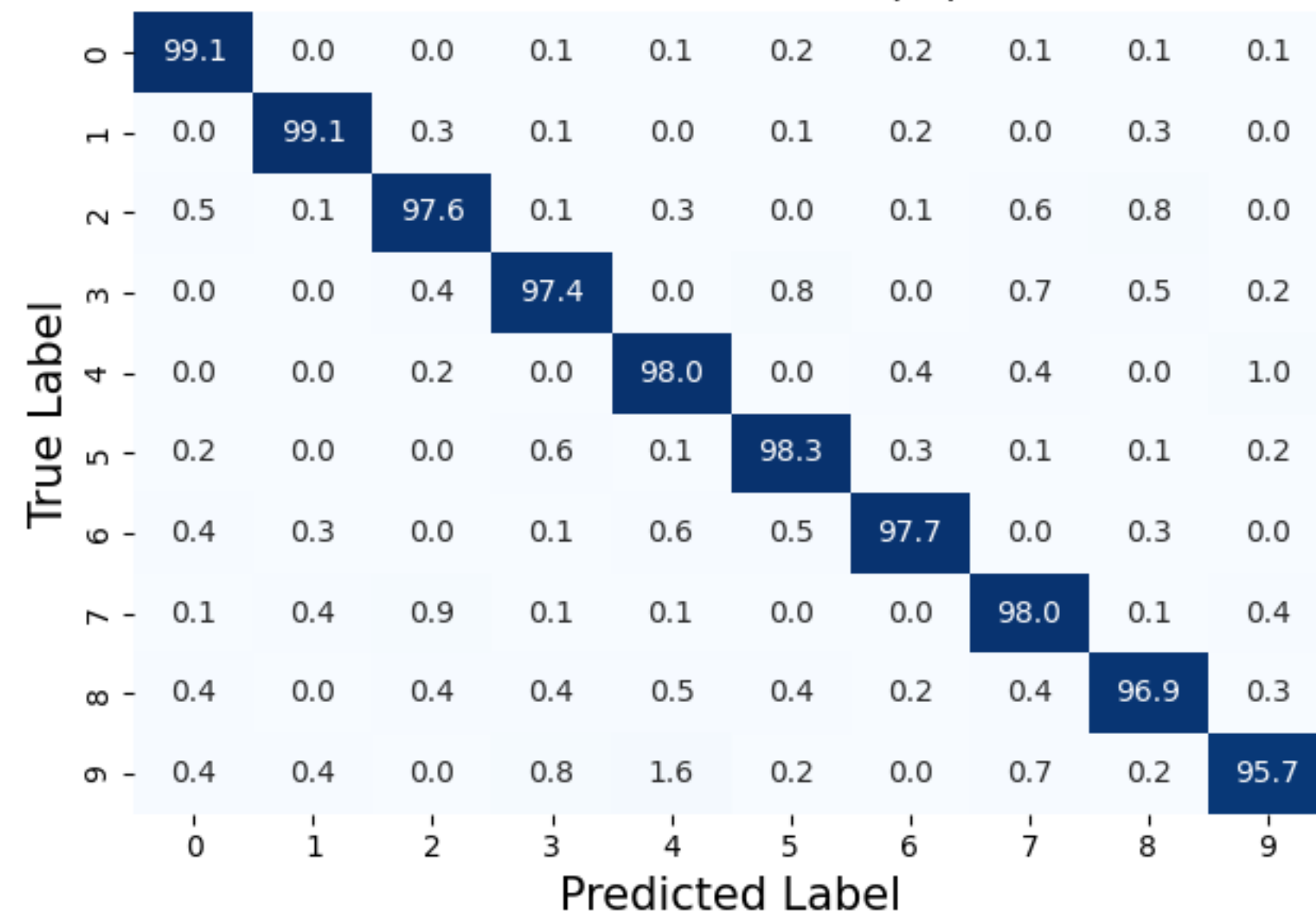
To us, these are the same digit.
The network has never been told that.

The Same Digit, Moved

The Network Does

Centered

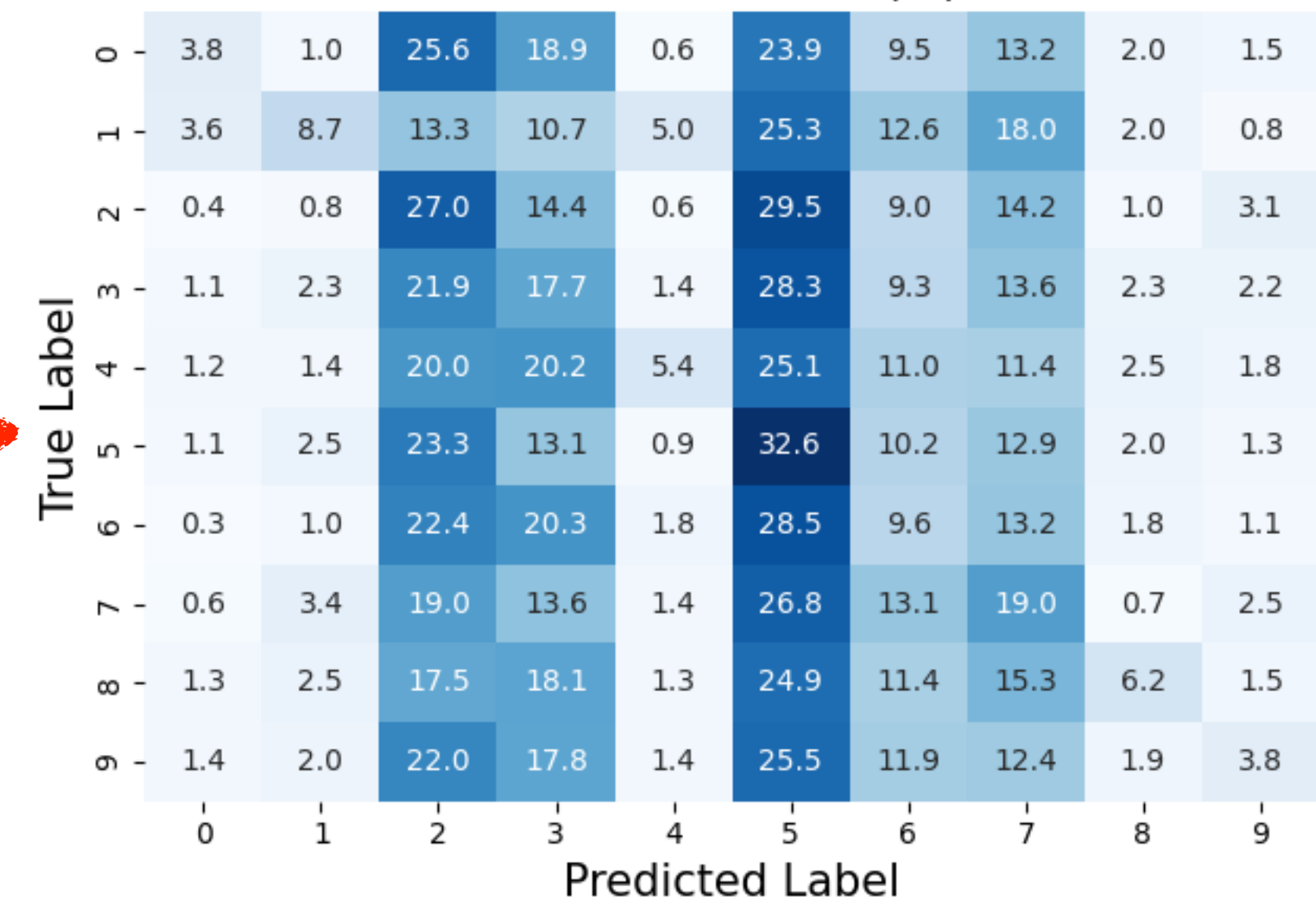
Confusion Matrix (%)



Collapse

Shifted

Confusion Matrix (%)



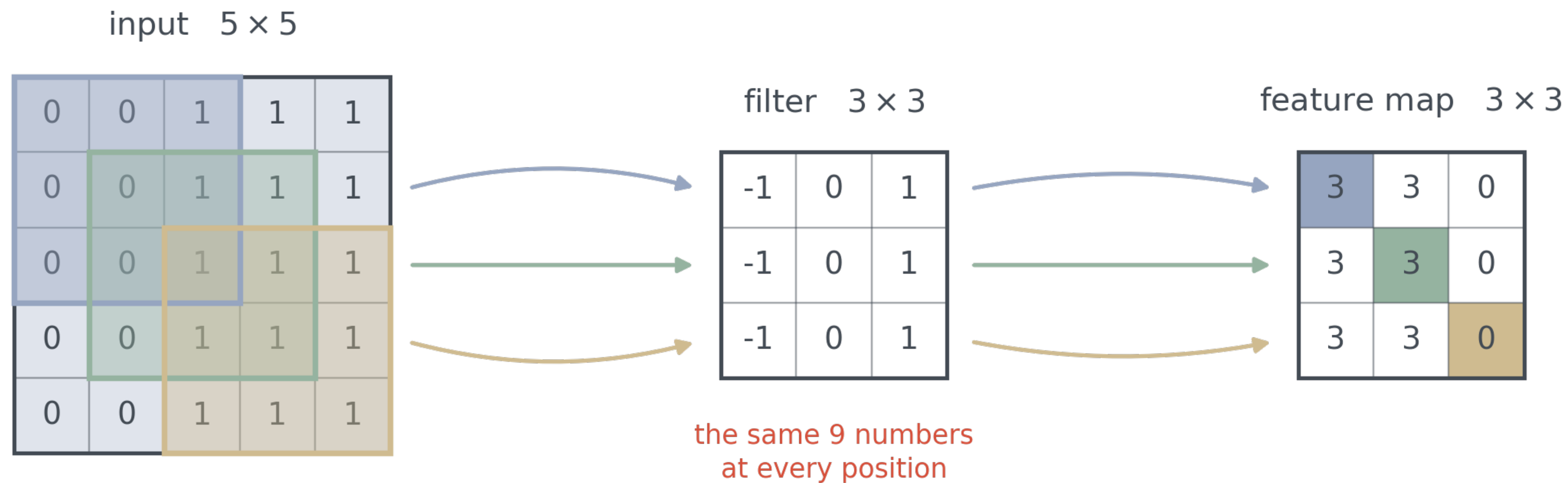
Inductive Bias

Let the Network Know What We Know

- A network that starts from nothing has to learn everything from the data – including facts we already know for certain.
- We know a 6 is a 6 wherever it sits in the frame. The network does not, and it had no way to find out: it never saw a shifted digit.
- **Inductive bias** is what we build into the architecture instead of hoping the data will teach it. **Weight sharing** says "the same feature, anywhere"; **pooling** says "roughly where, not exactly where".
- Every architecture is a claim about the problem. Convolution claims translation. The right question is never "which network is best" but "which symmetry does my problem have".

Operations in CNN — Convolution

What One Filter Computes



$$\text{one output cell} = \sum_{i,j} (\text{patch})_{ij} (\text{filter})_{ij}$$

e.g. $0 \cdot (-1) + 0 \cdot 0 + 1 \cdot 1$ three times $= 3$

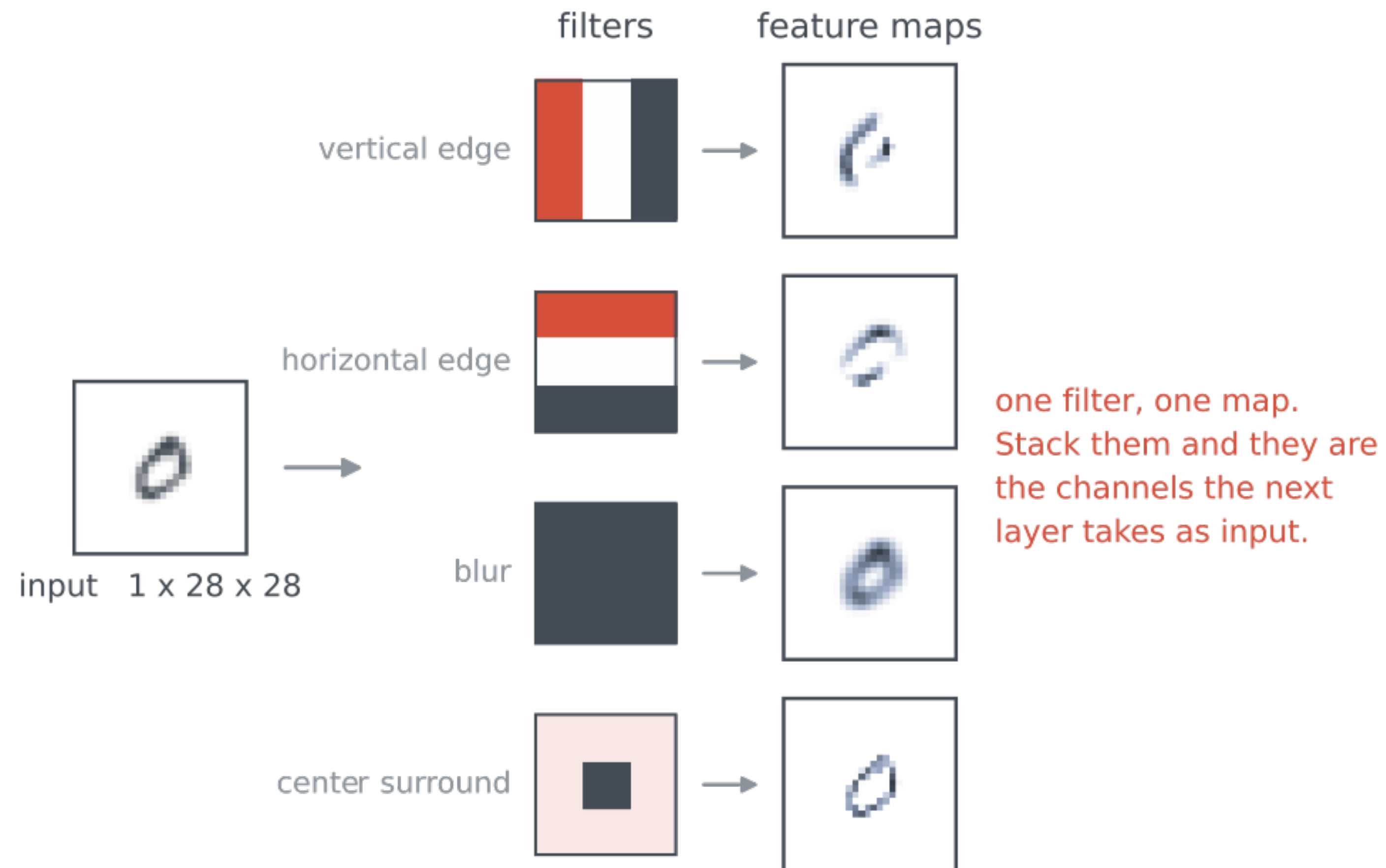
Operations in CNN — Weight Sharing

The Same Nine Numbers, Everywhere

- The filter does not change as it slides. The weights that find an edge at the top left are the same weights that find it at the bottom right. Fewer parameters than the fully connected network.
- It is also a statement about the world. Sharing weights across positions says "this pattern means the same thing wherever it appears" – and that is either true of your data, or it is not.
- For images, it is often approximately true for local image features. For a table where column 3 is missing energy and column 4 is a jet count, that is false; convolution is the wrong tool.

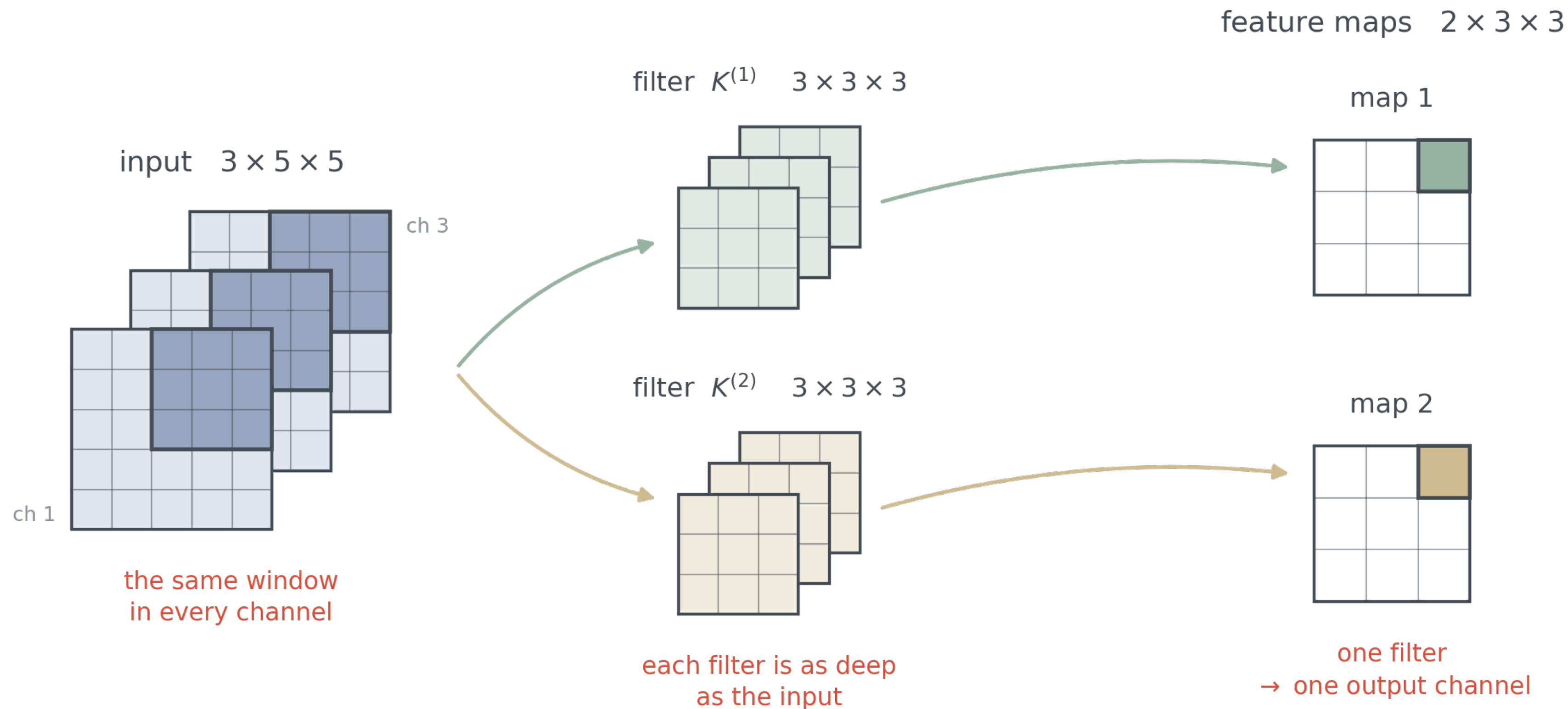
Operations in CNN — Channels

Many Filters, Read in Parallel



Operations in CNN — Channels

One Filter Per Output Channel



$$\text{one cell of map } k = \sum_c \sum_{i,j} (\text{patch}_c)_{ij} (K_c^{(k)})_{ij} + b^{(k)}$$

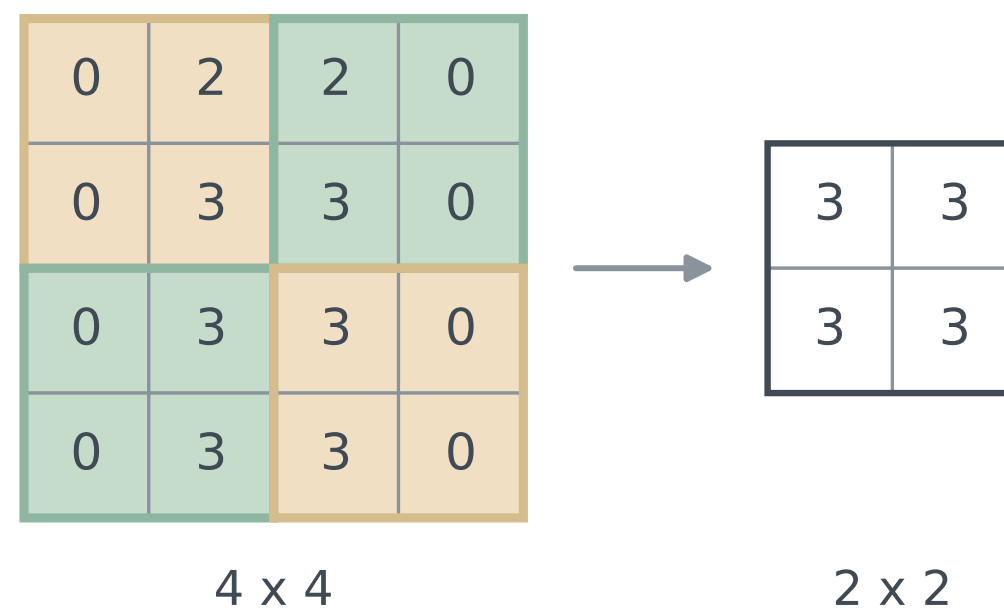
$$\text{parameters} = C_{\text{out}} \times C_{\text{in}} \times k \times k + C_{\text{out}} = 2 \times 3 \times 3 \times 3 + 2 = 56$$

Operations in CNN — Pooling

Summarizing a Neighborhood, for Free

Max pool 2

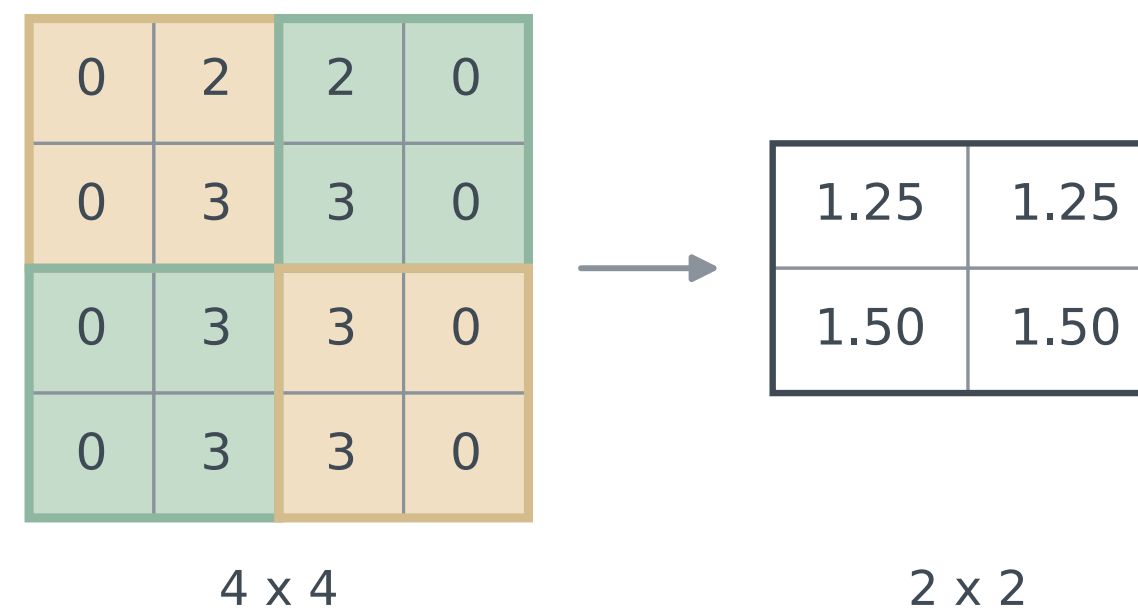
the largest value in each 2 x 2 block



Keeps 'there is a strong response somewhere in here'.
Drops 'it was exactly there'.

Average pool 2

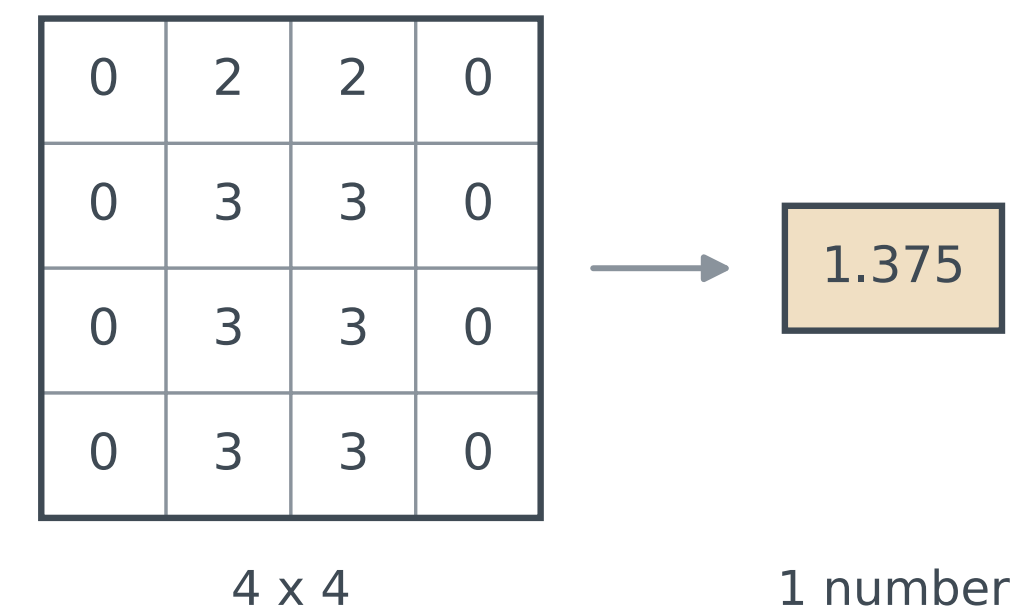
the mean of each 2 x 2 block



Keeps how much response there was, not just the peak. Smoother, and less willing to ignore the rest.

Global average pool

the mean of the whole map



One number per channel, whatever the map's size. This is the one our network ends with.

None of the three has a single learned parameter.

Dimensionality

What Decides the Output Size

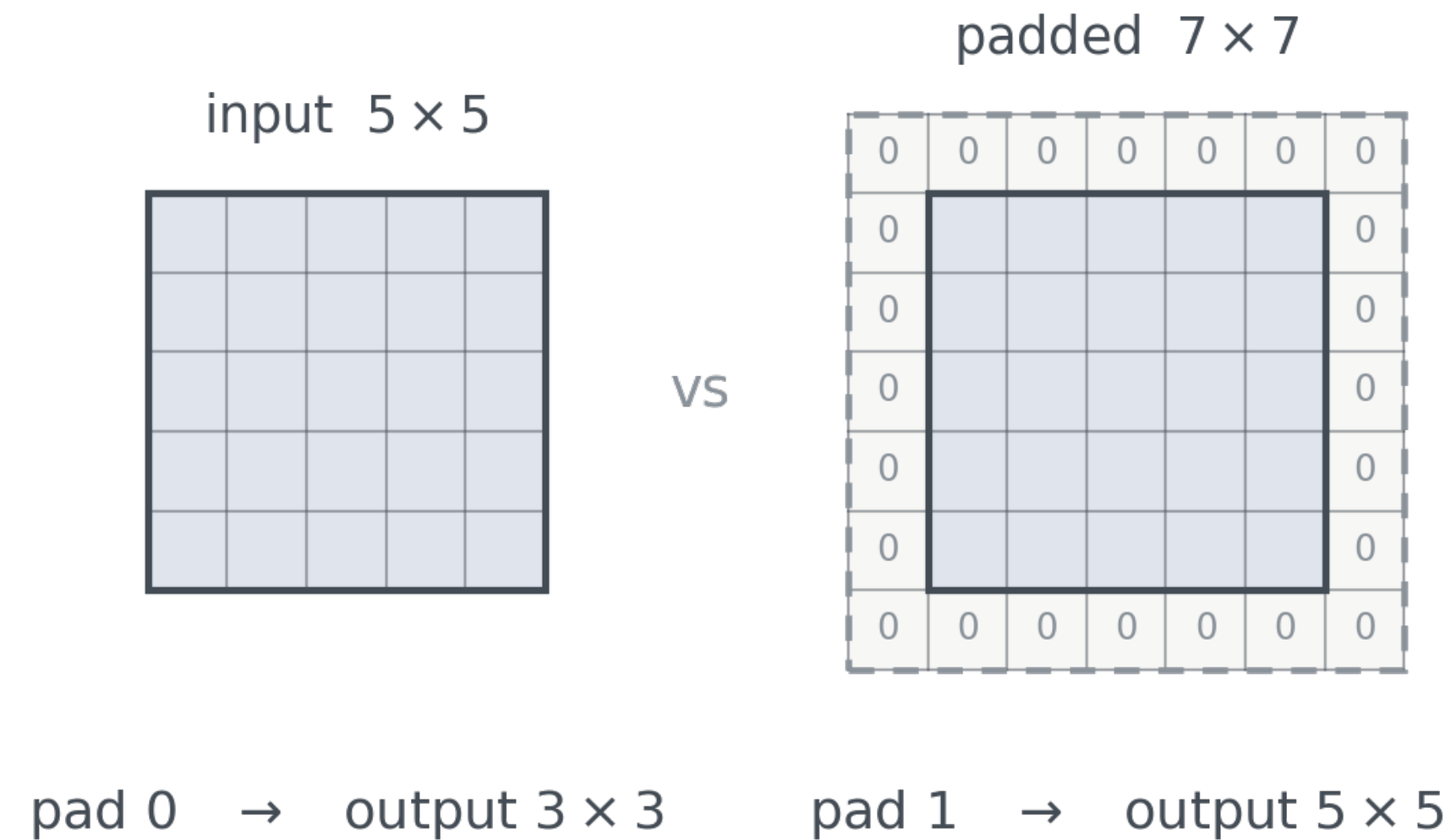
- Padding – a ring of zeros around the edge. A 3×3 filter shrinks each axis by 2; pad 1 puts them back, so convolution leaves the size alone.
- Stride – how far the filter jumps. Stride 2 keeps every other position and roughly halves the spatial size.
- Getting this arithmetic wrong is the most common way a network fails to run at all. It is worth doing on paper once.

Dimensionality

What Decides the Output Size

Padding

a ring of zeros around the edge



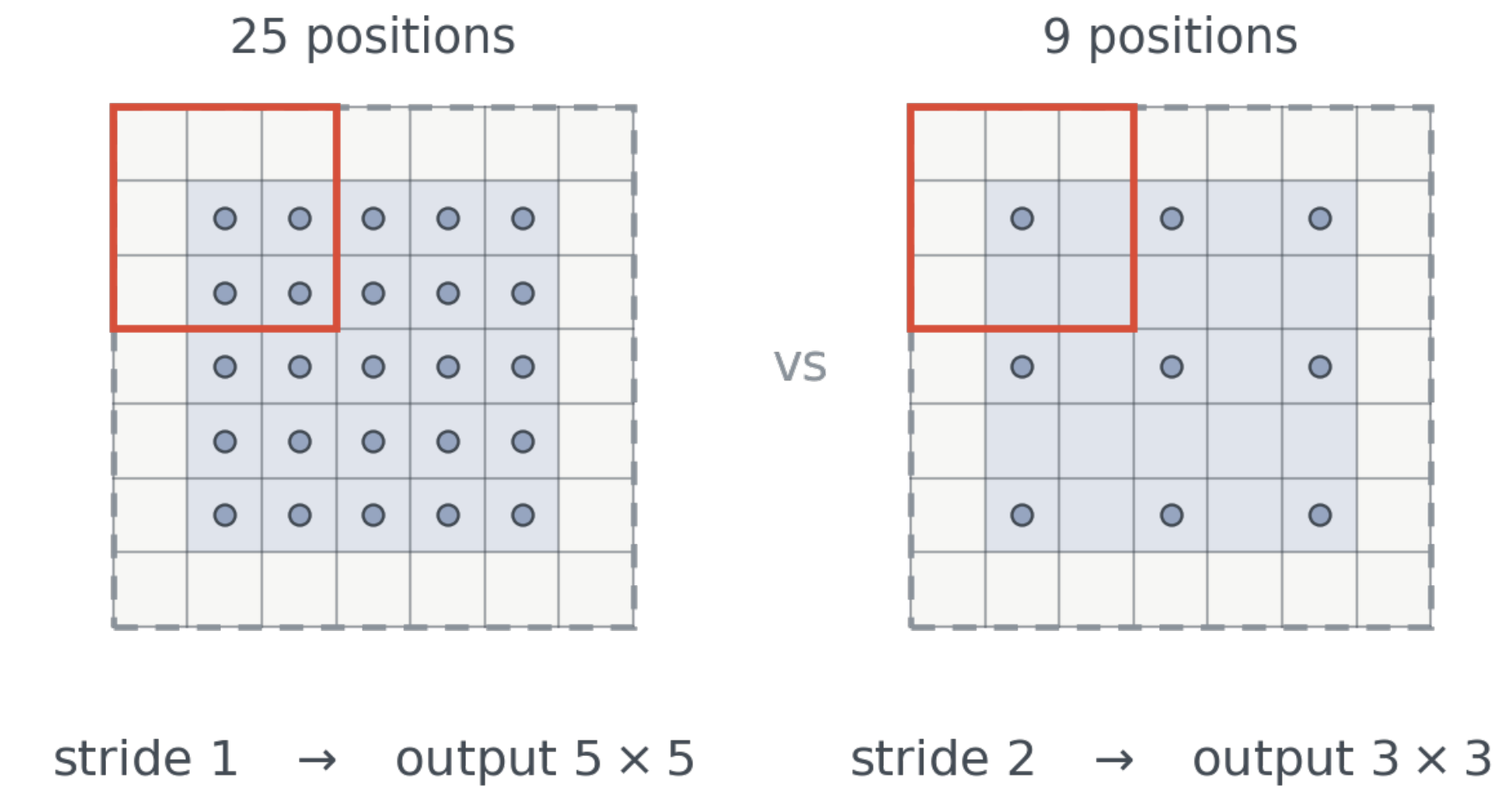
A 3×3 filter shrinks each axis by 2.
One ring of zeros gives them back.

$$\text{output size} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$$

n = input, k = filter, p = padding, s = stride

Stride

how far the filter jumps

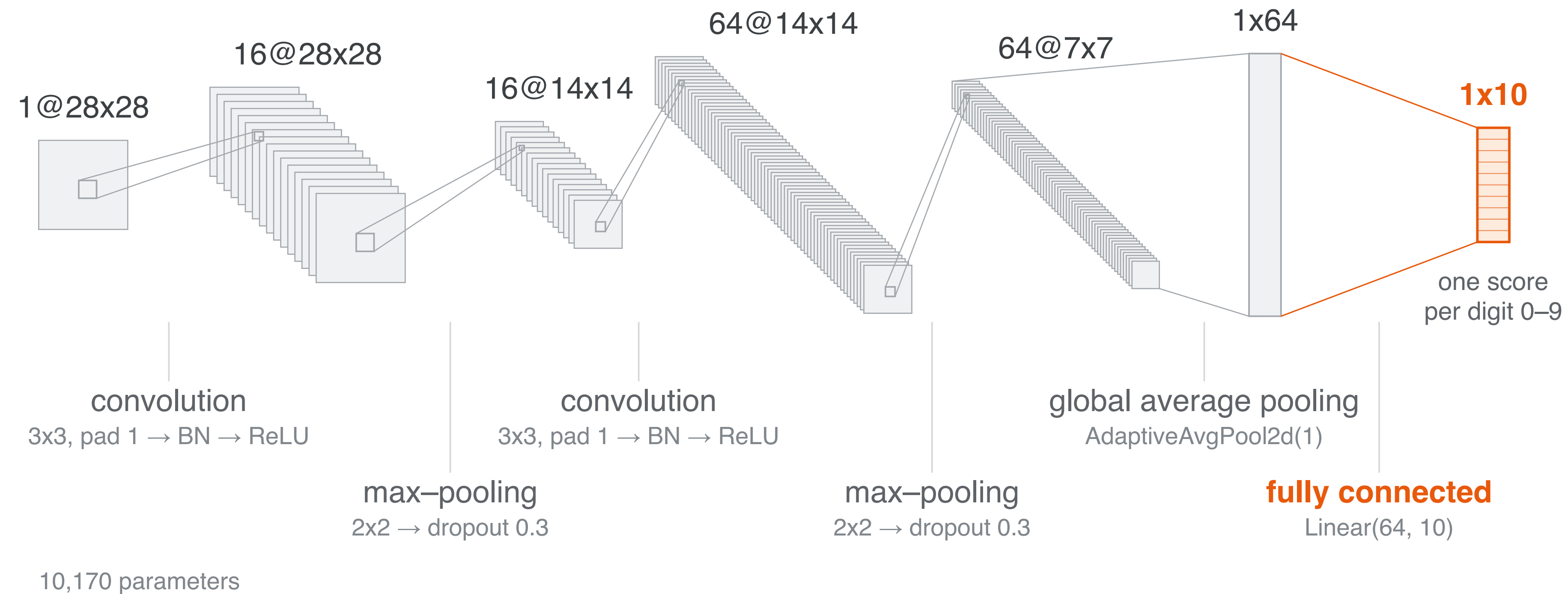


Every dot is one window position — and one output cell.
Stride 2 keeps every other one.

Basic Architecture

Stacking the Three Operations

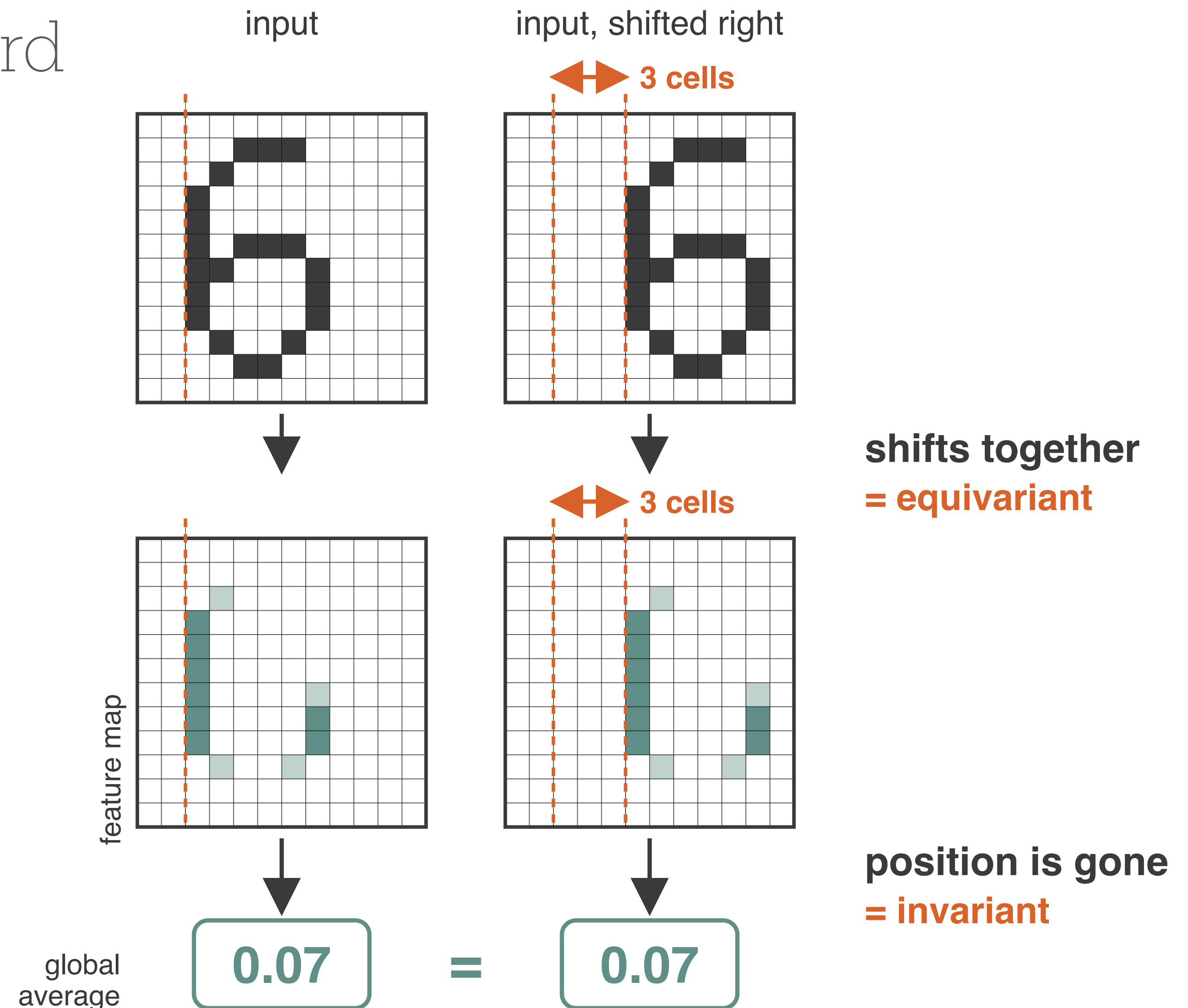
```
CNN(  
  (features): Sequential(  
    (0): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
    (1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (2): ReLU()  
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
    (4): Dropout2d(p=0.3, inplace=False)  
    (5): Conv2d(16, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
    (6): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (7): ReLU()  
    (8): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
    (9): Dropout2d(p=0.3, inplace=False)  
  )  
  (gap): AdaptiveAvgPool2d(output_size=1)  
  (classifier): Linear(in_features=64, out_features=10, bias=True)  
)  
number of parameters: 10,170
```



Equivariance and Invariance

Why This Head Became Standard

- Convolution: shift in $\rightarrow$ same shift out (**equivariant**)
- "Which digit", not "where"
- Global average pooling: position gone (**invariant**)

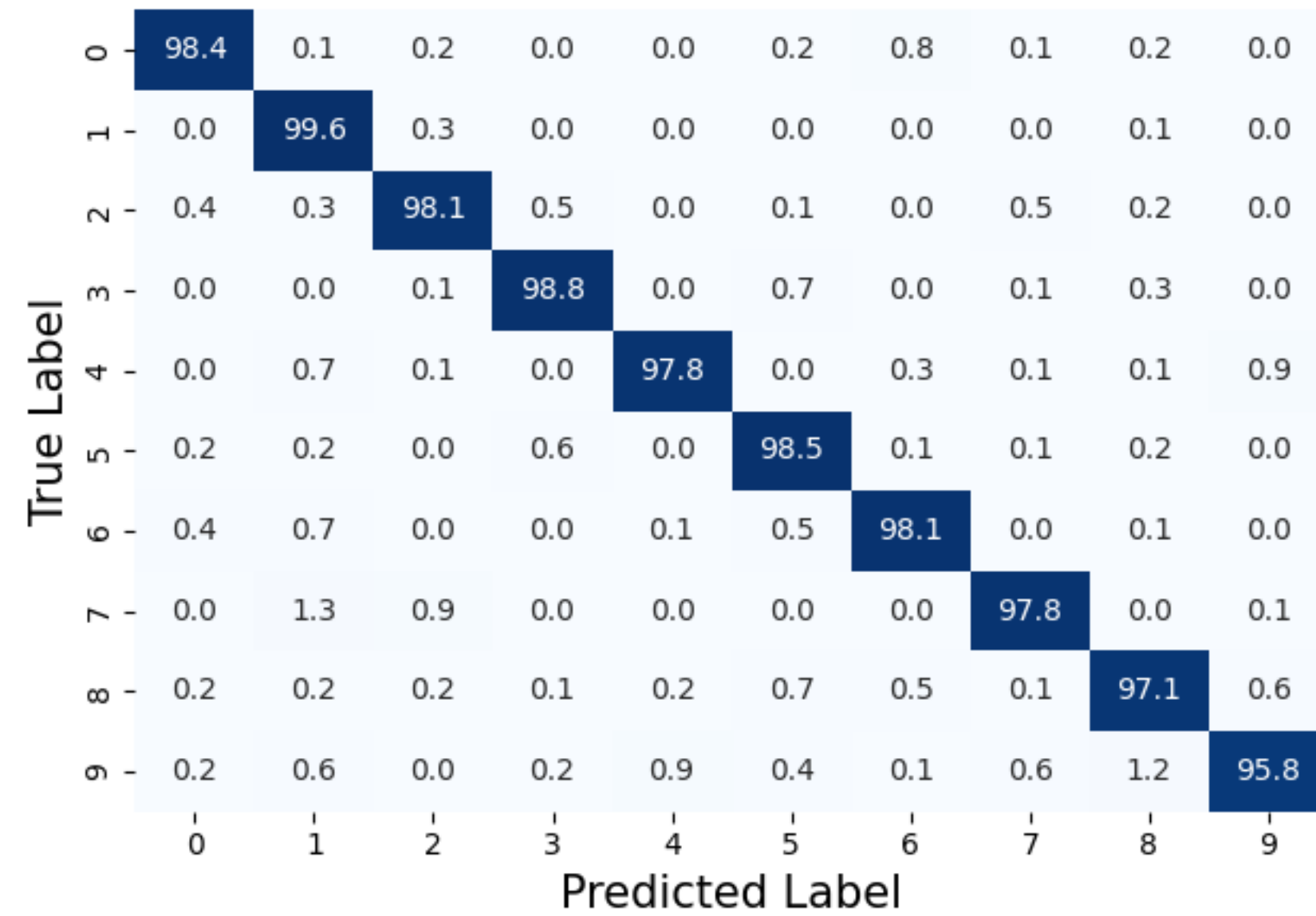


Results

Almost Untouched by the Shift

Centered

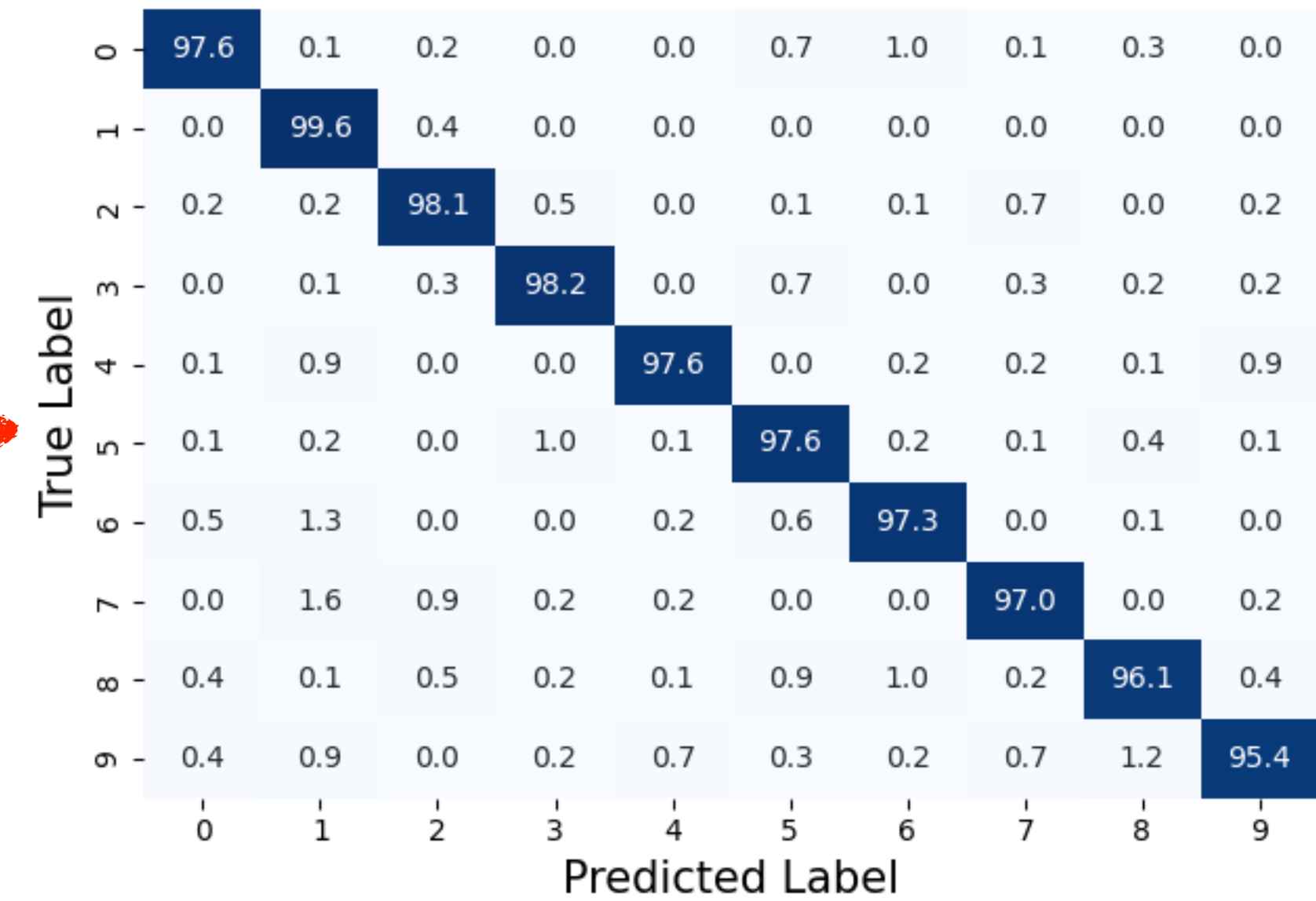
Confusion Matrix (%)



Preserved

Shifted

Confusion Matrix (%)



Epilogue

So... what did we actually learn?

Same big idea:
Build a function, measure how wrong it is,
and make it less wrong. 😊

1 Data

We start with data
(e.g., images).



MNIST

2 Perceptron

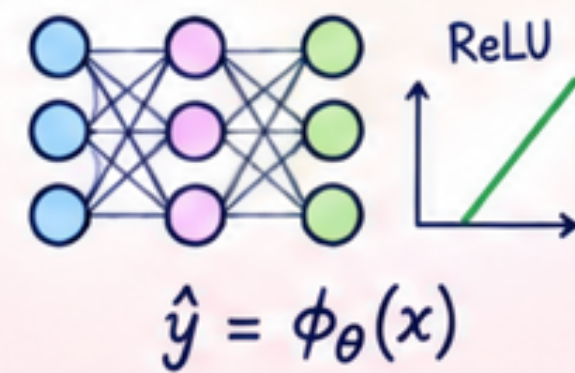
A single neuron
computes a linear
combination.

$$z = w^T x + b$$



3 MLP + Nonlinearity

Stacking neurons and
nonlinear activations
lets us model complex
functions.



4 Loss

We measure how wrong
our predictions are.
e.g., cross-entropy loss
 $\mathcal{L}(\theta)$



5 Optimization

We update the parameters
to reduce the loss
(e.g., gradient descent).

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$



6 Better Model

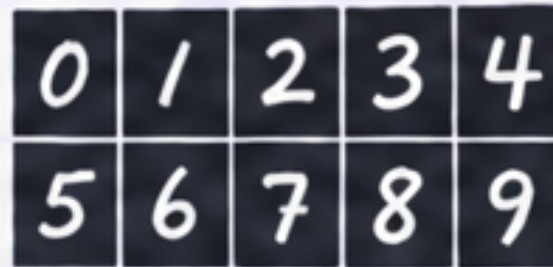
Repeat the process
to get a better model
that generalizes.



7 Hands-on: MNIST Classification

We build and train a neural network
and classify handwritten digits.

It works!
I can recognize
digits!



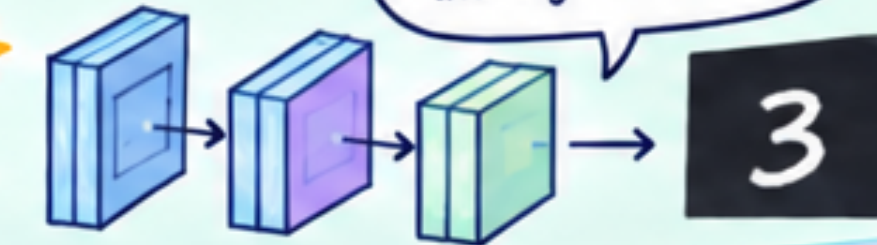
8 A challenge: shifted images

When the digit is shifted, a plain MLP
struggles to generalize.



9 CNN to the rescue!

Convolutional Neural Networks use
local patterns (filters) and are more
robust to translations.



I can find
the digit anywhere!

Key Takeaways

- ✓ A neural network is a (learnable) function.
- ✓ Nonlinearity is essential.
- ✓ We train by minimizing a loss with optimization.
- ✓ Watch the validation loss — the training loss lies.
- ✓ Real data brings new challenges (e.g., shifts).
- ✓ CNNs can leverage structure in data.
- ✓ You can build and train these models yourself! 😊

Wow!
It all connects!



Same building blocks.
Infinite possibilities. 😊

Told you we'd
learn together!



This is just
the beginning! 😊